

4

Il processore

In un problema importante, nessun dettaglio è insignificante.

Proverbio francese

4.1 Introduzione

Nel Capitolo 1 abbiamo visto che le prestazioni di un calcolatore sono determinate da tre fattori chiave: numero di istruzioni, durata del ciclo di clock e *numero di cicli di clock per istruzione* (CPI). Il compilatore e l'architettura dell'insieme delle istruzioni, che abbiamo esaminato nel Capitolo 2, determinano il numero di istruzioni richieste da un dato programma. Tuttavia, sia il periodo di clock sia il numero di cicli di clock per istruzione dipendono dalla particolare implementazione del processore. In questo capitolo, costruiremo un'unità di elaborazione dati (*datapath*) e l'unità di controllo (*control unit*) associata a due differenti implementazioni hardware dell'insieme di istruzioni RISC-V.

In particolare, il capitolo fornisce una spiegazione dei principi e delle tecniche adottate nella costruzione di un processore, partendo, in questo paragrafo, da un livello di astrazione elevato e da una visione semplificata. Passeremo poi a descrivere passo dopo passo come ideare un'unità di elaborazione e una versione semplificata di processore sufficienti a implementare un insieme di istruzioni come quello del MIPS. Nella parte centrale del capitolo descriveremo un'implementazione più realistica del RISC-V, basata sulla **pipeline**. Infine, seguirà il Paragrafo 4.11 nel quale saranno sviluppati i concetti necessari a implementare insiemi di istruzioni più complessi, come l'x86.

Per i lettori interessati all'interpretazione ad alto livello delle istruzioni e del loro impatto sulle prestazioni di un programma, i concetti che stanno alla base della pipeline sono presentati in questo paragrafo iniziale e nel Paragrafo 4.6. Gli sviluppi più recenti sono descritti nel Paragrafo 4.11, mentre il paragrafo



PIPELINE

successivo è dedicato alla descrizione delle architetture Core i7 Intel e Cortex-A8 ARM. Nel Paragrafo 4.13 viene mostrato come sfruttare il parallelismo a livello di istruzioni per raddoppiare le prestazioni della funzione che effettua la moltiplicazione di due matrici, discussa nel Paragrafo 3.9. Questi paragrafi consentono di comprendere ad alto livello il funzionamento di una pipeline.

I Paragrafi 4.3, 4.4 e 4.7 saranno particolarmente utili ai lettori più interessati al funzionamento del processore e alle sue prestazioni, mentre coloro che vogliono comprendere come si costruisce un processore dovrebbero leggere anche i Paragrafi 4.2, 4.8–4.10. Per i lettori interessati alla progettazione moderna dell'hardware, il Paragrafo 4.14  descrive i linguaggi di progettazione dell'hardware e gli strumenti CAD impiegati per implementare l'hardware e per descrivere l'implementazione di una pipeline. Questo paragrafo, inoltre, propone altri esempi di come siano eseguite le istruzioni dall'hardware basato su pipeline.

Un'implementazione di base del RISC-V

Esamineremo ora un'implementazione che comprende il seguente sottoinsieme di istruzioni di base del RISC-V:

- le istruzioni di riferimento alla memoria *load word* (lw) e *store word* (sw);
- le istruzioni aritmetico-logiche add, sub, and e or;
- le istruzioni di salto condizionato dal risultato di un test di uguaglianza, *branch if equal* (beq).

Un tale sottoinsieme non comprende tutte le istruzioni sugli interi (per es., sono assenti le istruzioni di scorrimento, moltiplicazione e divisione) e non include neppure alcuna istruzione sui numeri in virgola mobile, ma consente di illustrare i principi chiave utilizzati per costruire un'unità di elaborazione e per progettare l'unità di controllo relativa. L'implementazione delle restanti istruzioni è molto simile.

Nell'esaminare l'implementazione, avremo l'opportunità di comprendere come l'architettura dell'insieme delle istruzioni determini molti aspetti dell'implementazione stessa, e come la scelta tra le varie strategie di implementazione influenzi sia la frequenza di clock sia il CPI del calcolatore. Molti principi chiave introdotti nel Capitolo 2, quale *La semplicità favorisce la regolarità*, trovano diretta applicazione nell'implementazione delle istruzioni. Inoltre, molti concetti introdotti in questo capitolo per implementare il sottoinsieme di istruzioni RISC-V sono gli stessi su cui si basa la realizzazione della maggior parte dei calcolatori, dai server ad alte prestazioni ai microprocessori universali, ai processori embedded.

Una panoramica sull'implementazione

Nel Capitolo 2 abbiamo visto l'insieme di base delle istruzioni RISC-V, comprese le istruzioni aritmetico-logiche sugli interi, le istruzioni di accesso alla memoria e le istruzioni di salto condizionato. Molti componenti necessari per realizzare queste istruzioni sono gli stessi, indipendentemente dal tipo di istruzione. Per ogni istruzione i primi due passi sono identici.

1. Inviare il contenuto del *program counter* (PC) alla memoria che contiene il programma e prelevare l'istruzione dalla memoria; questa operazione viene anche indicata con il termine inglese di *fetch*, letteralmente “raggiungere”.
2. Leggere il contenuto di uno o due registri utilizzando i campi dell'istruzione per selezionare i registri. Per l'istruzione *load word* (lw) abbiamo bisogno di leggere un solo registro, ma la maggior parte delle altre istruzioni richiede la lettura di due registri.

Dopo questi due passi, le azioni richieste per completare l'esecuzione delle istruzioni dipendono dalla loro tipologia. Fortunatamente, per ognuna delle tre classi (istruzioni di accesso alla memoria, aritmetico-logiche e di salto), le azioni da compiere sono, a grandi linee, le stesse, indipendentemente dal particolare codice operativo dell'istruzione. La semplicità e la regolarità dell'insieme di istruzioni RISC-V semplificano l'implementazione rendendo simile l'esecuzione di istruzioni di classi diverse.

Per esempio, tutti i tipi di istruzioni, eccetto i salti incondizionati, utilizzano l'unità aritmetico-logica (ALU) dopo aver letto i registri; le istruzioni di accesso alla memoria utilizzano la ALU per il calcolo dell'indirizzo, le istruzioni aritmetico-logiche per l'esecuzione della loro operazione e i salti condizionati per effettuare i confronti. Dopo avere utilizzato la ALU, le azioni richieste per completare l'esecuzione variano a seconda del tipo di istruzione. Un'istruzione di accesso alla memoria avrà bisogno di accedere alla memoria sia per scrivere i dati (con una store) sia per leggerli (mediante una load). Un'istruzione aritmetico-logica, invece, dovrà scrivere i dati prodotti dalla ALU nel registro destinazione. Infine, in un'istruzione di salto condizionato l'indirizzo dell'istruzione successiva potrà cambiare in base al risultato del confronto; in assenza di salto, il PC verrà incrementato di 4 per puntare all'istruzione successiva.

La Figura 4.1 mostra uno schema ad alto livello di astrazione dell'implementazione di un RISC-V, focalizzato sulle varie unità funzionali e sulle loro interconnessioni. Sebbene questa figura mostri che la maggior parte del flusso dei dati "scorre" attraverso il processore, omette due importanti aspetti dell'esecuzione di un'istruzione.

Per prima cosa, in diversi punti, la Figura 4.1 mostra dati che provengono da due diverse sorgenti e arrivano alla stessa unità funzionale. Per esempio, il valore scritto nel PC può provenire da uno dei due sommatori ("Add" nella

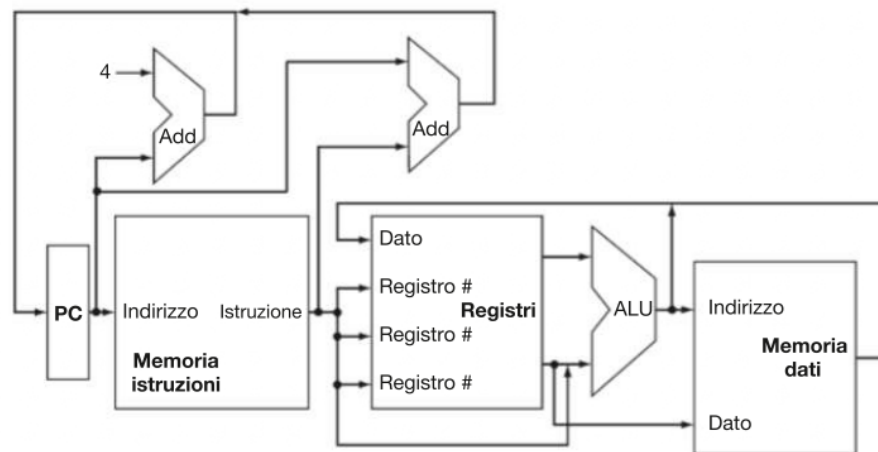


Figura 4.1 Uno schema astratto dell'implementazione di un sottoinsieme delle istruzioni RISC-V che mostra le principali unità funzionali e le principali connessioni tra loro. Tutte le istruzioni iniziano utilizzando il PC per fornire il loro indirizzo alla memoria istruzioni. Dopo che l'istruzione è stata caricata (*fetch*), il numero d'ordine dei registri contenenti gli operandi può essere letto nei campi opportuni dell'istruzione stessa. Una volta caricati gli operandi, si può elaborare il loro contenuto per determinare un indirizzo di memoria (nel caso di load o store), per eseguire un calcolo effettivo (operazioni aritmetico-logiche su interi) o per eseguire un confronto (salto condizionato). Se l'istruzione è di tipo aritmetico-logico, il risultato dalla ALU deve venire scritto in un registro. Se l'operazione è una load o una store, il risultato della ALU viene utilizzato come indirizzo, rispettivamente, per leggere dalla memoria il dato da scrivere nel register file o per scrivere il contenuto di un registro in memoria. I salti condizionati richiedono l'utilizzo dell'uscita della ALU per determinare l'indirizzo dell'istruzione successiva; tale indirizzo può provenire dalla ALU, nella quale PC e offset vengono sommati, oppure da un sommatore apposito che incrementa il PC di 4. Le linee che collegano tra loro le unità funzionali rappresentano i bus, formati da gruppi di segnali di controllo. Le frecce indicano la direzione in cui l'informazione scorre. Dato che le linee dei segnali possono incrociarsi senza necessariamente essere connesse, la connessione tra due linee viene esplicitamente segnalata con un punto.

figura); il dato scritto nel register file può provenire sia dalla ALU sia dalla memoria. Inoltre, il secondo input della ALU può provenire da un registro o dal campo immediato dell'istruzione. In pratica, queste linee multiple per i dati non possono semplicemente essere connesse tra loro, ma dobbiamo introdurre un circuito logico che selezioni da quale, fra le differenti sorgenti collegate, debba essere preso il dato, connettendo la sorgente opportuna con la sua destinazione. Questa selezione viene comunemente effettuata con un dispositivo chiamato *multiplexer*, che funge da "selettore dei dati" (*data selector*).

Il circuito logico del multiplexer viene descritto in dettaglio nell'Appendice A ; esso porta in uscita il contenuto di uno degli ingressi a seconda del valore presente sulle sue linee di controllo. Le linee di controllo sono impostate principalmente in base alle informazioni prelevate dall'istruzione stessa che viene eseguita.

Il secondo aspetto omesso nella figura è che i segnali di controllo di molte unità funzionali presenti nello schema dipendono dal tipo di istruzione. Per esempio, la memoria dati deve essere letta da una load, mentre deve essere scritta da una store. Il register file deve essere scritto solamente nel caso di un'istruzione di load o di un'istruzione aritmetico-logica. Naturalmente, la ALU deve poi poter scegliere l'operazione corretta tra le tante che ha a disposizione, come abbiamo visto nel Capitolo 2 (l'Appendice A  descrive in dettaglio la progettazione dei circuiti della ALU). Come nei multiplexer, le linee di controllo impostate sulla base del contenuto dei vari campi dell'istruzione dirigono l'elaborazione.

La Figura 4.2 mostra l'unità di elaborazione della Figura 4.1 integrata con i tre multiplexer richiesti e con le linee di controllo per le principali unità funzionali. L'unità di controllo, che ha come ingresso l'istruzione, viene utilizzata per determinare come impostare le linee di controllo per le unità funzionali e per due dei tre multiplexer. Il terzo multiplexer, che determina se nel PC debba essere scritto il valore $PC + 4$ o l'indirizzo di destinazione del salto, viene controllato dall'uscita "Zero" della ALU; questa uscita è impostata a 1 quando l'uguaglianza tra i due operandi è verificata, e quindi riporta il risultato del confronto richiesto dall'istruzione beq. La regolarità e la semplicità dell'insieme delle istruzioni RISC-V consentono di utilizzare un processo di decodifica semplice per impostare le linee di controllo.

Nel resto del capitolo raffineremo questo schema introducendo dei dettagli che richiederanno l'aggiunta di altre unità funzionali, l'aumento del numero di connessioni tra le unità, e, naturalmente, il miglioramento dell'unità di controllo affinché essa sia in grado di stabilire quali azioni debbano essere compiute per ogni diverso tipo di istruzione. I Paragrafi 4.3 e 4.4 descrivono una semplice implementazione che utilizza un singolo ciclo di clock, molto lungo, per ogni istruzione ed è basata sulla forma generale dell'unità di elaborazione descritto nelle Figure 4.1 e 4.2. In questa CPU ogni istruzione inizia l'esecuzione su un fronte del segnale di clock e completa l'esecuzione sul fronte di clock successivo.

Sebbene sia facile da comprendere, questo approccio non è pratico, dal momento che il ciclo di clock deve essere allungato di molto per dare il tempo alle istruzioni più lente di terminare l'esecuzione. Dopo aver progettato l'unità di controllo di questo semplice calcolatore, esamineremo implementazioni più veloci con tutte le loro complessità, compresa la gestione delle eccezioni.

Autovalutazione

Quanti dei cinque componenti classici di un calcolatore (unità di controllo, processore, memoria, input e output) sono mostrati nelle Figure 4.1 e 4.2?

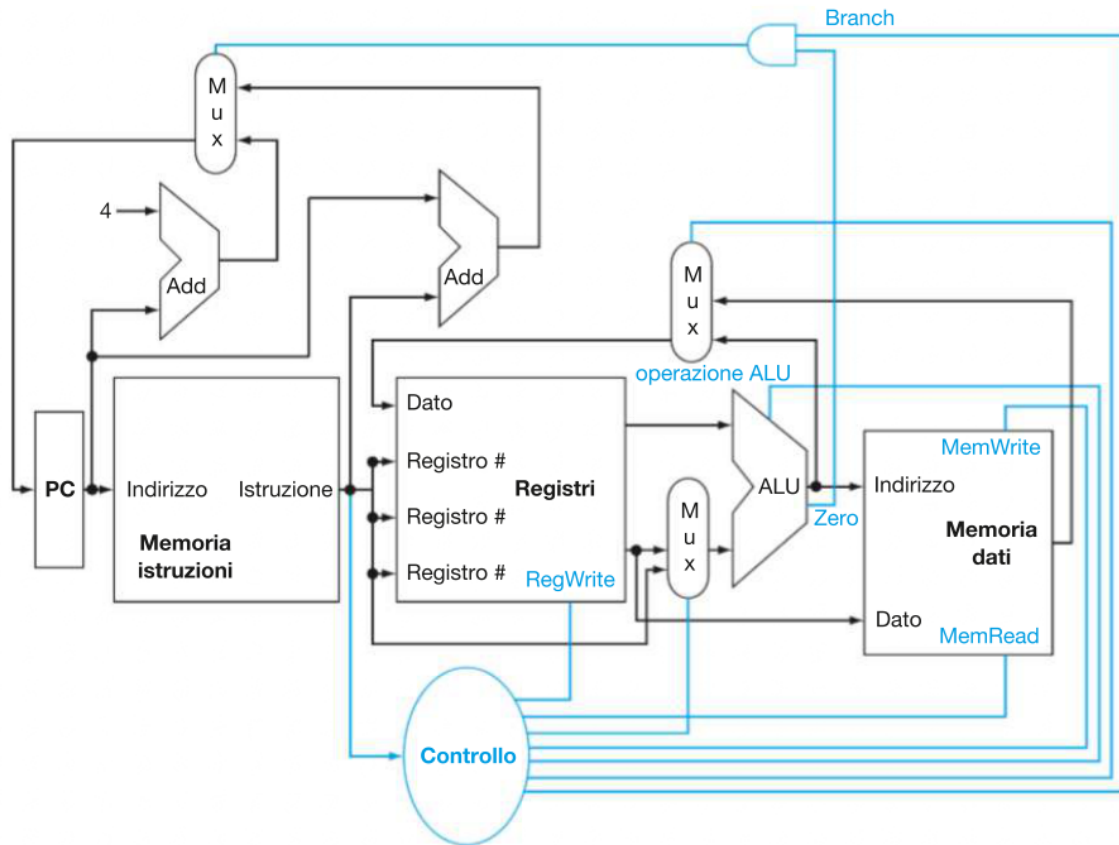


Figura 4.2 Implementazione base del sottoinsieme di istruzioni RISC-V, comprendente anche i multiplexer necessari e le linee di controllo.

Il multiplexer "Mux" in alto controlla ciò che viene scritto nel PC: $PC + 4$ oppure l'indirizzo di destinazione del salto condizionato; questo multiplexer viene controllato da una porta logica che esegue l'AND tra il segnale di Zero in uscita dalla ALU e un segnale di controllo che indica che l'istruzione è un'istruzione di salto condizionato. Il multiplexer posizionato centralmente nella parte alta della figura, la cui uscita ritorna al register file, viene utilizzato per portare l'uscita della ALU (in caso di istruzione aritmetico-logica) oppure l'uscita della memoria dati (in caso di load) al register file per la scrittura. Infine l'ultimo multiplexer, posizionato più in basso, viene utilizzato per determinare se il secondo ingresso della ALU provenga dai registri (per un'istruzione aritmetico-logica oppure per un'istruzione di salto condizionato) o dal campo costante dell'istruzione stessa (per un'operazione di load o store). Le linee di controllo aggiunte hanno un significato evidente e determinano le operazioni che devono essere eseguite dalla ALU oppure se la memoria debba essere letta o scritta o ancora se si debba eseguire un'operazione di scrittura sul register file. Le linee di controllo sono mostrate in blu in modo da risultare più visibili.

4.2 Convenzioni del progetto logico

Nell'analizzare il progetto di un calcolatore, dobbiamo decidere come funzionerà la logica hardware che implementa il calcolatore e come esso dovrà essere sincronizzato. Questo paragrafo riprende alcuni concetti fondamentali di logica digitale che verranno utilizzati largamente in tutto il capitolo. Il lettore che non abbia una buona preparazione nel campo della progettazione dei circuiti digitali, troverà utile leggere l'Appendice A prima di proseguire oltre.

Gli elementi funzionali che costituiscono l'unità di elaborazione del RISC-V sono costituiti da due diverse classi di elementi logici: elementi che operano sui dati ed elementi che contengono lo stato. Gli elementi che operano sui dati sono di tipo **combinatorio**, il che significa che in ogni istante i loro output dipendono solamente dagli input ricevuti nello stesso istante. Dati gli stessi valori di ingresso, un elemento combinatorio produrrà sempre il medesimo valore in uscita. La ALU nella Figura 4.1, e descritta nell'Appendice A, costituisce un esempio di elemento combinatorio: dato un certo insieme di valori in ingresso, essa produce sempre la stessa uscita perché non possiede alcun elemento interno di memoria.

Elemento combinatorio: un elemento in grado di eseguire delle operazioni, per esempio una porta AND o una ALU.

Gli elementi dell'unità di elaborazione che non sono combinatori contengono lo *stato*. Un elemento può contenere lo stato solo se ha al suo interno elementi di memoria. Chiameremo questi **elementi di stato**, perché se spegniamo e riaccendiamo il calcolatore, possiamo far ripartire il flusso di lavoro del calcolatore esattamente dallo stesso punto caricando gli elementi di stato con i valori che essi contenevano prima di avere spento. Inoltre, salvando e ripristinando gli elementi di stato, il calcolatore si comporterebbe come se non fosse mai stato spento. Di conseguenza, gli elementi di stato caratterizzano completamente il funzionamento del calcolatore. Nella Figura 4.1 la memoria istruzioni, la memoria dati e i registri sono esempi di elementi di stato.

Un elemento di stato possiede almeno due ingressi e un'uscita. Gli ingressi richiesti sono il valore da scrivere nell'elemento e il clock, che determina quando scrivere. L'uscita di un elemento di stato è il valore contenuto al suo interno, scritto in un ciclo di clock passato. Per esempio, uno degli elementi di stato più semplici, dal punto di vista logico, è il *flip-flop* di tipo D (vedi Appendice A ) che ha esattamente questi due ingressi (il valore del dato e il clock) e un'uscita. Oltre ai flip-flop, l'architettura RISC-V utilizza altri due tipi di elementi di stato: le memorie e i registri, che compaiono anch'essi nella Figura 4.1. Il clock viene utilizzato per determinare quando l'elemento di stato deve essere scritto, mentre la lettura può avvenire in qualsiasi istante.

I componenti logici che contengono lo stato sono anche detti *sequenziali*, in quanto le loro uscite dipendono sia dagli ingressi sia dal valore del loro stato interno. Per esempio, l'uscita dell'unità funzionale che contiene i registri dipende sia dal numero del registro specificato sugli ingressi sia dal valore contenuto da quel particolare registro, che era stato scritto in precedenza. L'Appendice A  descrive in modo dettagliato il funzionamento e la struttura degli elementi combinatori e di quelli sequenziali.

Metodologia di temporizzazione

La **metodologia di temporizzazione** definisce quando i segnali possono essere scritti e quando possono essere letti. È importante temporizzare le operazioni di lettura e scrittura, perché se un segnale venisse letto e contemporaneamente scritto, il valore letto potrebbe corrispondere al valore precedente la scrittura, a quello successivo, o addirittura a una combinazione dei due. Ovviamente i progettisti dei calcolatori non possono accettare questa imprevedibilità nei risultati, che viene evitata proprio grazie alla metodologia di temporizzazione.

Per semplicità, supporremo di utilizzare una metodologia di **temporizzazione sensibile ai fronti** (*edge-triggered*); essa garantisce che il valore memorizzato all'interno di un elemento logico sequenziale venga aggiornato solamente in corrispondenza di un fronte del segnale di clock, che è una veloce transizione dal livello basso al livello alto e viceversa (Figura 4.3). Poiché solo gli elementi di stato possono memorizzare i dati, un qualsiasi circuito combinatorio deve ricevere l'input da un insieme di elementi di stato e può scrivere il risultato dell'elaborazione in un secondo insieme di elementi di stato. Gli ingressi sono i valori che erano stati scritti in un ciclo di clock precedente, mentre gli output del circuito combinatorio sono valori che potranno essere utilizzati in un ciclo di clock successivo. La Figura 4.3 mostra un blocco di logica combinatoria posto tra due elementi di stato. Esso lavora all'interno di un singolo ciclo di clock: tutti i segnali si devono propagare a partire dall'elemento di stato 1 e devono attraversare la logica combinatoria fino a raggiungere l'elemento di stato 2 nel tempo di un periodo di clock. Il tempo richiesto perché i segnali raggiungano l'elemento di stato 2 definisce quindi la durata minima del periodo di clock. Per semplicità, non si riporterà esplicitamente il **segnale di controllo** della scrittura per quegli elementi di stato che vengono scritti

Elemento di stato: un elemento di memoria, per esempio un registro o una memoria.

Metodologia di temporizzazione: l'approccio utilizzato per determinare quando un dato è valido e stabile in relazione al segnale di clock.

Temporizzazione sensibile ai fronti: una tecnica di temporizzazione nella quale tutti i cambiamenti di stato avvengono su un fronte del segnale di clock.

Segnale di controllo: un segnale utilizzato per la selezione in un multiplexer o per dirigere il funzionamento di un'unità funzionale; si distingue dai segnali che portano i dati, che contengono l'informazione che viene elaborata dalle unità funzionali.

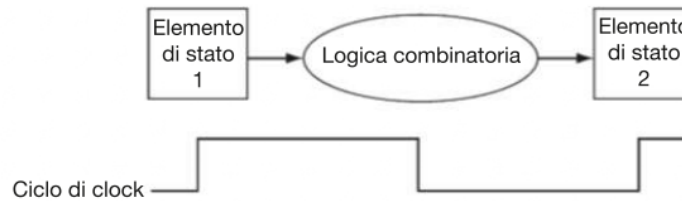


Figura 4.3 Logica combinatoria, elementi di stato e clock sono strettamente correlati. In un sistema digitale sincrono, il segnale di clock determina quando gli elementi di stato scrivono la loro memoria interna. Gli ingressi a un elemento di stato devono raggiungere un valore stabile (cioè devono aver raggiunto un valore che non cambierà almeno fino a dopo il fronte di clock successivo) prima che il fronte attivo del clock provochi l'aggiornamento dello stato. In questo capitolo, supporremo che tutti gli elementi di stato, comprese le memorie, siano sensibili al fronte di salita, cioè cambino contenuto quando il clock passa dallo stato basso allo stato alto.

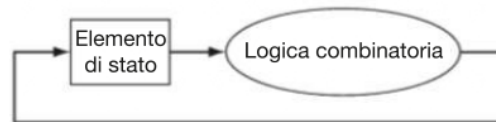


Figura 4.4 Una metodologia sensibile ai fronti permette di leggere e scrivere un elemento di stato nello stesso ciclo di clock, senza creare una competizione che potrebbe causare indeterminazione nel contenuto dell'elemento. Ovviamente, il ciclo di clock deve avere una durata sufficiente a garantire che gli ingressi siano stabili quando arriva il fronte attivo del clock. Non si può verificare alcuna retroazione all'interno del singolo ciclo di clock, grazie all'aggiornamento sul fronte del clock dell'elemento di stato. Se fosse possibile aggiornare l'elemento di stato prima del fronte di clock, il circuito non potrebbe funzionare correttamente. I circuiti presentati in questo capitolo e nel prossimo si basano sulla metodologia di temporizzazione sensibile ai fronti e su strutture simili a quella mostrata in questa figura.

a ogni fronte attivo del clock. Viceversa, se un elemento di stato non viene aggiornato a ogni ciclo di clock, allora si renderà necessario un segnale esplicito di controllo della scrittura. Sia il segnale di clock sia il segnale di controllo della scrittura sono ingressi, e il contenuto dell'elemento di stato può essere modificato solamente sul fronte attivo del clock, quando cioè il segnale di controllo della scrittura è asserito.

Asserito: un segnale associato a un livello logico alto o vero.

Deasserito: un segnale associato a un livello logico basso o falso.

Utilizzeremo il termine **asserito** per indicare un segnale che si trova nello stato logico alto e il termine *asserire* per indicare che un segnale deve essere portato al livello logico alto; utilizzeremo il termine **non asserito** per indicare un segnale che si trova nello stato logico basso e il termine *non asserire* per indicare che un segnale deve essere portato al livello logico basso. Abbiamo introdotto i termini “asserire” e “non asserire” perché un 1 può rappresentare a volte un valore logico alto e altre volte un valore logico basso.

La metodologia sensibile ai fronti permette di leggere il contenuto di un registro, inviare il valore attraverso uno o più blocchi di logica combinatoria e scrivere lo stesso registro nello stesso ciclo di clock, come mostrato nella **Figura 4.4**. Non occorre specificare se la scrittura avvenga sul fronte di salita (da basso ad alto) o di discesa (da alto a basso) del segnale di clock, dal momento che gli ingressi a un circuito logico combinatorio possono cambiare solo sul fronte attivo del clock. In questo testo utilizzeremo il fronte di salita del clock. Con la metodologia di temporizzazione sensibile ai fronti non si corre il rischio di innescare una retroazione all'interno dello stesso ciclo di clock, e il circuito logico illustrato nella **Figura 4.4** funziona correttamente. Nell'Appendice A  sono discussi brevemente alcuni vincoli temporali aggiuntivi (come i tempi di set-up e di hold) e vengono presentate altre metodologie di temporizzazione.

Quasi tutti gli elementi di stato e combinatori dell'architettura RISC-V a 32 bit hanno ingressi e uscite di ampiezza pari a 32 bit, essendo questa l'ampiezza della maggior parte dei dati elaborati dal processore. Quando incontreremo

unità con ingressi o uscite di ampiezza diversa da 32 bit la loro presenza sarà indicata esplicitamente. Nelle figure, i bus, ossia le linee che trasportano segnali di ampiezza maggiore di 1 bit, saranno indicati con linee più spesse. In alcuni casi due o più bus verranno uniti per formare un bus di ampiezza maggiore; per esempio, possiamo ottenere un bus da 32 bit unendo due bus da 16 bit. In questi casi, etichette sulle linee indicheranno chiaramente che due bus sono stati concatenati per ottenere un unico bus. Verranno inoltre aggiunte delle frecce per indicare la direzione del flusso dei dati tra gli elementi. Infine, un diverso colore consentirà di distinguere i segnali di controllo dai dati, distinzione che diverrà più chiara procedendo con la lettura del capitolo.

Autovalutazione

Vero o falso? Poiché il register file può essere letto e scritto nello stesso ciclo di clock, le unità di elaborazione RISC-V che utilizzano la metodologia di temporizzazione sensibile ai fronti devono possedere più di una copia del register file.

Approfondimento. Esiste anche una versione dell'architettura RISC-V a 64 bit. Naturalmente, la maggior parte dei cammini interni di questa architettura è a 64 bit.

4.3 Realizzazione di un'unità di elaborazione

Un modo ragionevole per iniziare la costruzione di un'unità di elaborazione consiste nell'esaminare i componenti principali che servono per eseguire i diversi tipi di istruzioni RISC-V. Procediamo dall'inizio, analizzando quali **elementi dell'unità di elaborazione** (*datapath elements*) siano richiesti da ciascuna classe di istruzioni per poi approfondire i diversi elementi procedendo attraverso i diversi livelli di **astrazione**. Quando mostreremo i diversi elementi dell'unità di elaborazione, mostreremo anche i relativi segnali di controllo associati a quel livello di astrazione.

La **Figura 4.5a** mostra il primo elemento di cui abbiamo bisogno: un'unità di memoria in cui salvare le istruzioni del programma che sia in grado di fornire in uscita l'istruzione associata all'indirizzo dato in ingresso. La **Figura 4.5b** mostra anche il **program counter (PC)** che, come abbiamo già visto nel Capitolo 2, è un registro utilizzato per memorizzare l'indirizzo dell'istruzione corrente. Infine, è necessario un sommatore per incrementare di 4 il PC e ottenere l'indirizzo dell'istruzione successiva. Tale sommatore è un elemento combinatorio e può essere costruito a partire dalla ALU descritta in dettaglio nell'Appendice A ; è sufficiente impostare i segnali di controllo della ALU in modo che venga specificata sempre un'operazione di somma. Identificheremo negli schemi circuitali una ALU di questo tipo, configurata permanentemente come sommatore, con l'etichetta *Add*, come mostrato nella **Figura 4.5c**; essa non può eseguire le altre operazioni tipiche di una ALU.

Per eseguire una qualsiasi istruzione occorre anzitutto prelevare l'istruzione stessa dalla memoria. Per prepararsi a eseguire l'istruzione successiva, bisogna incrementare il program counter in modo che punti a quell'istruzione, cioè 4 byte dopo. La **Figura 4.6** mostra come si possono combinare i tre elementi della **Figura 4.5** per formare quella parte dell'unità di elaborazione che preleva le istruzioni e incrementa il PC per ottenere l'indirizzo dell'istruzione successiva del programma.



ASTRAZIONE

Elemento dell'unità di elaborazione: un'unità funzionale utilizzata per eseguire operazioni o memorizzare i dati all'interno del processore. Nell'implementazione RISC-V, gli elementi dell'unità di elaborazione sono la memoria dati, la memoria istruzioni, il register file, l'unità aritmetico-logica (ALU) e i sommatore.

Program counter (PC): il registro che contiene l'indirizzo dell'istruzione in esecuzione di un programma.

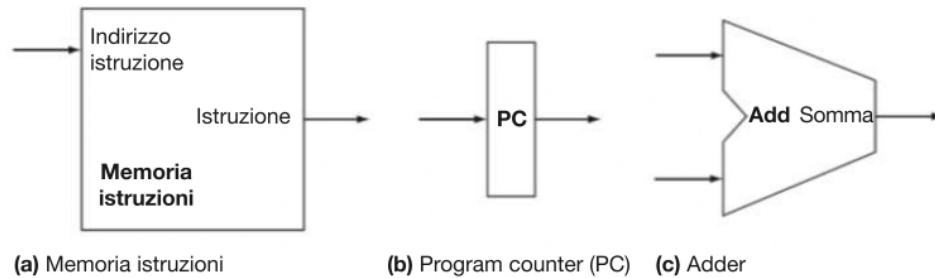


Figura 4.5 Sono necessari due elementi di stato per memorizzare e accedere alle istruzioni, e serve un sommatore per calcolare l'indirizzo dell'istruzione successiva. Gli elementi di stato sono la memoria delle istruzioni e il program counter (PC). La memoria delle istruzioni fornisce solamente un accesso in lettura, perché essa non viene scritta dall'unità di elaborazione. Per questo tratteremo la memoria istruzioni come un circuito logico combinatorio: l'uscita in ogni istante corrisponde al contenuto della locazione specificata dall'indirizzo in ingresso, e non è necessario alcun segnale di controllo esplicito. La memoria delle istruzioni dovrà essere scritta solamente nella fase di caricamento del programma; non è difficile aggiungere questa funzionalità e quindi, per semplicità, in questo libro non la considereremo. Il program counter è un registro a 32 bit che viene scritto al termine di ogni ciclo di clock e non richiede quindi un segnale di controllo esplicito per la scrittura. Il sommatore è una ALU configurata in modo da eseguire sempre la somma dei suoi due ingressi su 32 bit e fornire in uscita il risultato della somma su 32 bit.

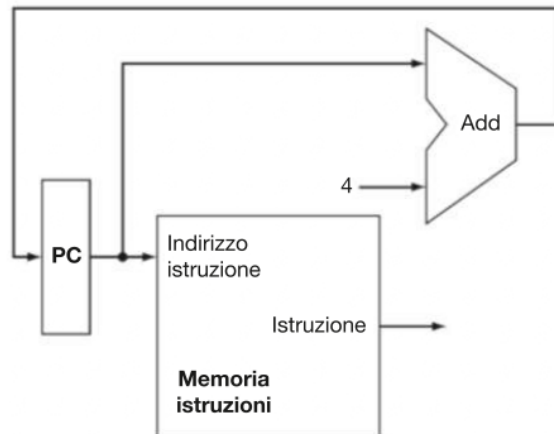


Figura 4.6 Parte dell'unità di elaborazione utilizzata per prelevare le istruzioni e incrementare il program counter. L'istruzione prelevata viene utilizzata dalle altre parti dell'unità di elaborazione.

Consideriamo ora le istruzioni in formato R (vedi Figura 2.19). Tutte le istruzioni di questo tipo leggono due registri, eseguono un'operazione con la ALU sul contenuto di questi due registri e, alla fine, scrivono il risultato in un registro. Chiameremo queste *istruzioni di tipo R* o *istruzioni aritmetico-logiche*, dato che eseguono operazioni aritmetiche o logiche. Questa classe di istruzioni comprende add, sub, and e or, introdotte nel Capitolo 2. Un tipico esempio di istruzione di questo tipo è add x1, x2, x3, che legge il contenuto di x2 e x3 e scrive il risultato dell'addizione nel registro x1.

I registri universali a 32 bit del processore sono raccolti in una struttura detta **register file** (banco di registri), un insieme di registri in cui ciascun registro può essere letto o scritto specificando il numero a esso associato all'interno dell'insieme. Il register file contiene lo stato dei registri del calcolatore. Avremo inoltre bisogno di una ALU per operare sui valori letti dai registri.

Poiché le istruzioni di tipo R hanno tre registri come operandi, per ciascuna istruzione dovremo leggere due dati di una parola ciascuno dal register file e poi scrivere il risultato, sempre nel register file. Per ogni parola letta dai registri è necessario prevedere un ingresso al register file che specifichi il *numero*

Register file: un elemento dello stato che contiene un insieme di registri che possono essere letti o scritti fornendo il numero del registro sul quale si vuole operare.

del registro da leggere e un'uscita dal register file sulla quale viaggerà il valore letto dal registro. Per scrivere un dato di una parola serviranno due ingressi: il primo ingresso deve specificare il *numero del registro di scrittura* e il secondo deve fornire il *dato* da scrivere nel registro. Il register file fornisce in qualunque momento in uscita il contenuto del registro il cui numero d'ordine è specificato sull'ingresso relativo al *numero del registro di lettura*. La scrittura, invece, viene controllata da un segnale di controllo esplicito, "RegWrite", che deve essere asserito perché il registro possa essere scritto in corrispondenza del fronte del clock. Quindi serviranno complessivamente quattro ingressi (tre per i numeri del registro e uno per il dato da scrivere) e due uscite (entrambe per i dati), come mostrato nella **Figura 4.7a**. Gli ingressi che specificano il numero dei registri hanno ampiezza di 5 bit, in modo da poter specificare uno dei 32 registri ($32 = 2^5$), mentre i bus dei dati in ingresso e in uscita sono di 32 bit ciascuno.

La **Figura 4.7b** mostra una ALU che riceve due input su 32 bit e produce un risultato su 32 bit; inoltre, produce un segnale a un bit che vale 1 se il risultato dell'operazione è 0. I segnali di controllo della ALU, su 4 bit, vengono descritti in dettaglio nell'Appendice A; li rivedremo brevemente quando dovremo definire il circuito che ne imposta il valore opportuno.

Consideriamo ora le istruzioni RISC-V di caricamento di un registro (load) e di trasferimento alla memoria (store). Esse hanno la forma generale `lw x1, offset(x2)` e `sw x1, offset(x2)`, e calcolano un indirizzo di memoria sommando il contenuto del registro base, in questo caso `x2`, al campo offset di 12 bit, contenuto nell'istruzione stessa, considerato dotato di segno. Se l'istruzione è una store, il dato da memorizzare deve essere letto dal register file dove risiede in `x1`, mentre se è una load, il valore letto dalla memoria deve essere scritto all'interno del register file nel registro specificato, in questo caso `x1`. Quindi, per eseguire queste istruzioni occorreranno sia il register file sia la ALU, come riportato nella **Figura 4.7**.

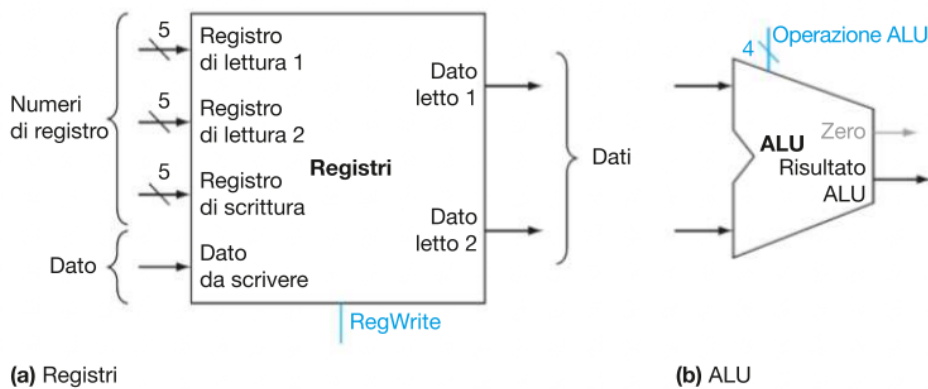


Figura 4.7 I due elementi richiesti per implementare le operazioni di tipo R sono il register file e la ALU. Il register file contiene tutti i registri e dispone di due porte di lettura e una di scrittura. Lo schema circuitale di un register file a più porte è discusso nel Paragrafo A.8 dell'Appendice A. In ogni istante il register file fornisce in uscita il contenuto dei registri corrispondenti agli ingressi "Registro di lettura", senza richiedere alcun segnale di controllo; viceversa, la scrittura di un registro deve essere indicata esplicitamente asserendo il segnale di controllo della scrittura, "RegWrite". Si ricordi che la scrittura avviene sul fronte attivo del clock, per cui tutti gli ingressi interessati (cioè il dato da scrivere, il numero di registro e il segnale di controllo) devono essere validi sul fronte attivo del clock. Essendo la scrittura attiva sui fronti, l'architettura implementata nel nostro progetto può tranquillamente leggere e scrivere lo stesso registro nello stesso ciclo di clock: la lettura fornirà il dato contenuto nel registro scritto in un ciclo di clock precedente, mentre il valore scritto potrà essere letto a partire dal ciclo di clock successivo. Gli ingressi che specificano al register file il numero d'ordine dei registri hanno ampiezza di 5 bit, mentre i bus che trasportano i dati sono ampi 64 bit. L'operazione che la ALU deve eseguire viene specificata dal segnale "Operazione ALU", che è ampio 4 bit per la ALU descritta nell'Appendice A. L'uscita "Zero" della ALU, verrà utilizzata tra breve per implementare i salti condizionati.

Estensione del segno: aumento della dimensione di un dato ottenuta replicando il bit più significativo, che rappresenta il segno del dato originale, nei bit più significativi del dato di destinazione.

Indirizzo di destinazione del salto: l'indirizzo che viene specificato in un salto condizionato; esso diventerà il nuovo indirizzo contenuto nel program counter (PC) se il salto condizionato viene eseguito. Nell'architettura RISC-V, l'indirizzo di destinazione di un salto è ottenuto mediante la somma del campo offset contenuto nell'istruzione stessa con l'indirizzo dell'istruzione di salto condizionato.

Inoltre, saranno necessarie anche un'unità per l'**estensione del segno** del campo offset su 12 bit, contenuto nell'istruzione, da estendere a 32 bit, con segno, e un'unità di memoria dati in cui scrivere o da cui leggere il dato. La memoria dati viene scritta dalle istruzioni di store, e sarà quindi dotata dei segnali di controllo sia di lettura sia di scrittura; riceverà in ingresso, inoltre, l'indirizzo e il dato che deve essere scritto. La **Figura 4.8** mostra questi due nuovi elementi.

L'istruzione beq ha tre operandi: due registri il cui contenuto viene confrontato per determinare se è uguale, e un offset di 16 bit utilizzato per calcolare l'**indirizzo di destinazione del salto** a partire dall'indirizzo dell'istruzione stessa. La forma di questa istruzione è beq x1, x2, offset; per eseguirla, occorre calcolare l'indirizzo di destinazione del salto, sommando il campo offset dell'istruzione (dopo averlo esteso a 32 bit con segno) al PC. Nella definizione delle istruzioni di salto condizionato ci sono due dettagli (Capitolo 2) ai quali è bene prestare attenzione.

- L'architettura dell'insieme delle istruzioni specifica che l'indirizzo di base per il calcolo dell'indirizzo di salto è quello dell'istruzione di salto stessa.
- L'architettura stabilisce anche che il campo offset sia spostato di 1 bit a sinistra, in modo tale che l'offset non codifichi lo spiazamento in numero di byte, ma in numero di mezze parole. Di fatto, tale spostamento aumenta lo spazio di indirizzamento dell'offset di un fattore 2 rispetto alla codifica dello spiazamento in byte.

Per poter gestire quest'ultima complicazione è necessario far scorrere il campo offset di 1 bit a sinistra.

Oltre a calcolare l'indirizzo di destinazione del salto, bisogna determinare se l'istruzione da eseguire dopo sia quella presente nella posizione di memoria successiva oppure quella contenuta all'indirizzo di destinazione del salto. Quando la condizione di salto è vera (cioè quando i due operandi sono uguali),

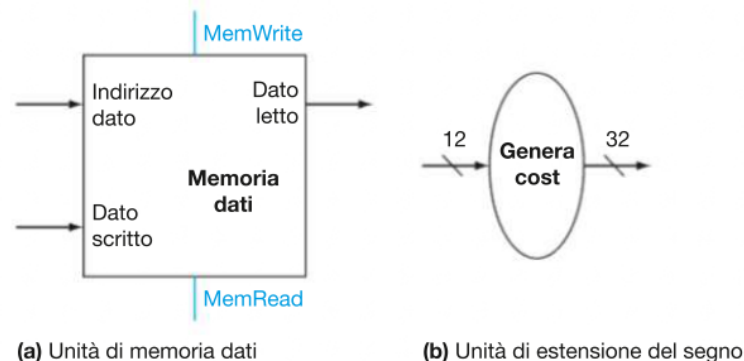


Figura 4.8 Le due unità necessarie per l'implementazione delle istruzioni di load e di store: oltre al register file e alla ALU mostrate nella **Figura 4.7**, sono richieste anche un'unità di memoria dati e un'unità di estensione del segno. L'unità di memoria è un elemento di stato che ha come ingressi l'indirizzo del dato e il dato da scrivere, e possiede una sola uscita per il dato letto. I segnali di controllo per la lettura e la scrittura sono separati, anche se all'interno di un ciclo di clock solo uno dei due può essere asserito. L'unità di memoria ha bisogno di un segnale di lettura esplicito, dal momento che, diversamente da quanto avviene per il register file, leggere il contenuto di un indirizzo non valido può causare dei problemi, come vedremo nel Capitolo 5. L'unità di generazione della costante riceve in ingresso un numero a 12 bit per una load, una store e un salto condizionato, e fornisce in uscita un valore esteso a 32 bit dotato di segno (Capitolo 2). Si suppone che la scrittura della memoria sia attiva su un fronte del segnale di clock. I chip di memoria standard, in realtà, possiedono un segnale di abilitazione alla scrittura che permette, quando asserito, l'esecuzione dell'operazione di scrittura. Sebbene il segnale di abilitazione alla scrittura nei chip di memoria reali non sia sensibile ai fronti, l'unità di elaborazione attiva sui fronti presentata in questo capitolo può essere facilmente adattata per funzionare anche con i chip di memoria reali. Per una discussione più approfondita sul funzionamento dei chip di memoria reali vedi Paragrafo A.8 dell'Appendice A.

l'indirizzo di destinazione del salto diventa il nuovo valore del PC e si parla di **salto preso** (*branch taken*, salto eseguito); se il contenuto degli operandi è diverso, il valore del PC incrementato di 4 sostituisce il valore corrente nel PC, proprio come in tutte le altre istruzioni; in questo caso si parla di **salto non preso** (*branch not taken*, salto non eseguito).

Per i salti condizionati, quindi, l'unità di elaborazione deve eseguire due operazioni: calcolare l'indirizzo di destinazione del salto e confrontare il contenuto di due registri. Inoltre, come si vedrà tra poco, i salti hanno anche un effetto sulla parte dell'unità di elaborazione incaricata di prelevare le istruzioni. La parte dell'unità di elaborazione che gestisce i salti è visualizzata nella **Figura 4.9**. Per il calcolo dell'indirizzo di destinazione, si utilizzano un'unità di estensione del segno, analoga a quella mostrata nella Figura 4.8, e un sommatore. Per eseguire il confronto possiamo utilizzare il register file mostrato nella Figura 4.7a, che ci permette di leggere il contenuto dei due registri contenenti gli operandi; osserviamo che, in questo caso, non è richiesto di scrivere il risultato nel register file. Il confronto tra i due operandi viene fatto dalla ALU descritta nell'Appendice A.

Dato che la ALU fornisce un segnale in uscita che indica se il risultato è pari a 0, si possono inviare i due operandi alla ALU specificando tramite i segnali di controllo di effettuare una sottrazione: se l'uscita Zero della ALU viene asserita, i due operandi sono uguali. Segnaliamo esplicitamente che, anche se l'uscita Zero indica sempre che il risultato di un'operazione è 0, noi la

Salto preso: un salto condizionato nel quale la condizione di salto è soddisfatta; in questo caso nel program counter (PC) viene scritto l'indirizzo di destinazione del salto. Le istruzioni di salto incondizionato possono essere assimilate a dei salti condizionati che vengono sempre eseguiti.

Salto non preso: un salto condizionato in cui la condizione di salto non è soddisfatta; in questo caso nel program counter (PC) viene scritto l'indirizzo dell'istruzione successiva all'istruzione di salto condizionato.

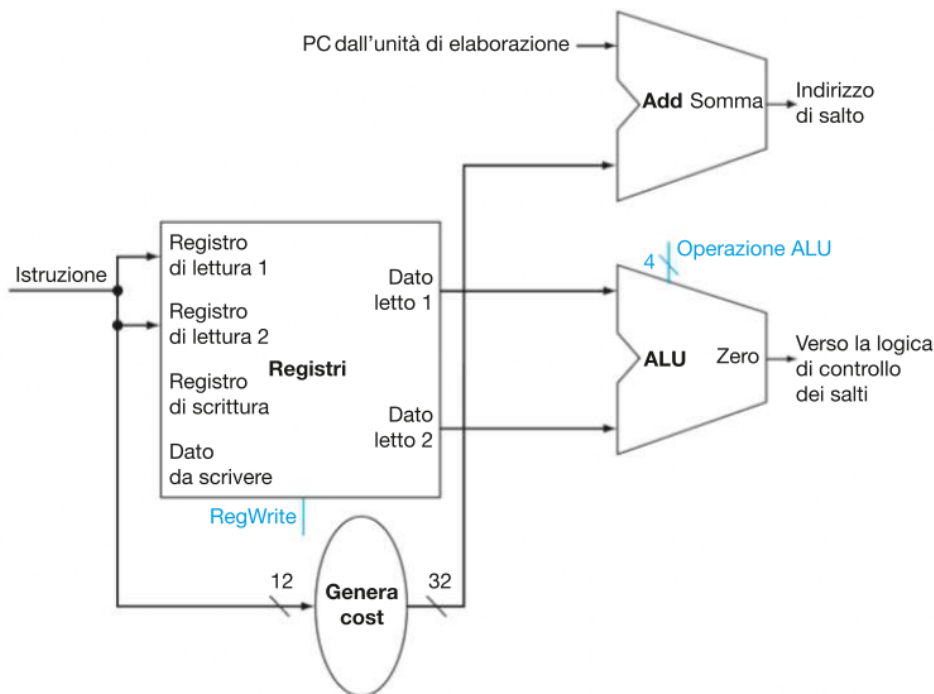


Figura 4.9 La porzione del cammino di elaborazione dei dati per i salti condizionati impiega la ALU per valutare la condizione di salto e un sommatore a parte per determinare l'indirizzo di destinazione del salto, calcolato come somma del PC e il contenuto dei 12 bit del campo immediato dell'istruzione (lo spiazzamento del salto), fatti scorrere a sinistra di 1 bit ed estesi a 32 bit con segno.

L'unità etichettata "Shift a sinistra 1" corrisponde semplicemente a un particolare instradamento dei segnali che rappresentano lo spiazzamento; l'instradamento è effettuato in modo tale che, a valle dell'estensione del segno, venga aggiunto un segnale uguale a 0_{due} all'estremità inferiore del campo offset: poiché l'entità dello spostamento è costante, non è richiesto alcun componente dedicato. Sapendo che lo spiazzamento viene esteso a partire dal 12° bit, questa operazione di scorrimento scarnerà solamente dei "bit di segno". La logica di controllo deciderà, in funzione dell'uscita Zero della ALU, se nel PC debba essere caricato il contenuto del PC incrementato di 4 oppure l'indirizzo di destinazione del salto.

utilizzeremo solamente per il test di uguaglianza dei salti condizionati. In seguito, vedremo precisamente come debbano essere collegati i segnali di controllo della ALU all'interno dell'unità di elaborazione.

L'istruzione di salto condizionato opera sommando al PC i 12 bit dell'istruzione fatti scorrere a sinistra di 1 posizione. Questo scorrimento si ottiene semplicemente concatenando degli 0 all'offset dell'istruzione di salto condizionato, come già descritto nel Capitolo 2.

Progettazione di un'unità di elaborazione unificata

Ora che abbiamo esaminato i componenti dell'unità di elaborazione necessari a eseguire i diversi tipi di istruzioni, possiamo combinarli in una singola unità di elaborazione aggiungendo i segnali di controllo opportuni. L'unità di elaborazione più semplice cercherà di eseguire tutte le istruzioni in un singolo ciclo di clock. Quindi, nessuna risorsa dell'unità di elaborazione può essere utilizzata più di una volta per ogni istruzione e ogni elemento funzionale utilizzato più di una volta deve essere duplicato: perciò servirà una memoria delle istruzioni separata da quella dei dati. Anche se dovremo duplicare alcune unità funzionali, molti elementi potranno essere condivisi dal flusso di esecuzione di istruzioni diverse.

La condivisione da parte di istruzioni di tipo diverso di uno stesso elemento potrebbe richiedere la presenza di collegamenti multipli agli ingressi di quell'elemento; in questo caso occorrerà introdurre un multiplexer, con i suoi segnali di controllo, per poter selezionare l'input corretto all'unità funzionale tra tutti gli input possibili.

ESEMPIO

Costruzione di un'unità di elaborazione

L'unità di elaborazione per le istruzioni aritmetico-logiche (o di tipo R) e quella per le istruzioni di accesso alla memoria sono molto simili. Le differenze principali sono le seguenti.

- Le istruzioni aritmetico-logiche utilizzano la ALU con gli ingressi provenienti da due registri. Le istruzioni di memoria utilizzano la ALU per calcolare l'indirizzo della memoria dati, ma prendono il secondo ingresso alla ALU dal campo offset a 12 bit dell'istruzione stessa, con l'estensione del segno.
- Il valore da scrivere nel registro destinazione proviene dalla ALU (per le istruzioni di tipo R) o dalla memoria (per l'istruzione load).

Si mostri come costruire un'unità di elaborazione relativa alla parte di calcolo per le istruzioni di accesso a memoria e per le istruzioni aritmetico-logiche, utilizzando un solo register file e una sola ALU per entrambi i tipi di istruzioni; si introducano i multiplexer necessari.

SOLUZIONE

Per creare un'unità di elaborazione che utilizzi un solo register file e una sola ALU, dobbiamo implementare un meccanismo che possa prelevare il secondo operando della ALU da due sorgenti diverse; lo stesso dobbiamo fare per il dato da scrivere nel register file.

Occorre quindi inserire un primo multiplexer all'ingresso della ALU e un secondo multiplexer all'ingresso del register file. La Figura 4.10 mostra la parte di esecuzione dell'unità di elaborazione che supporta questi due tipi di istruzioni.

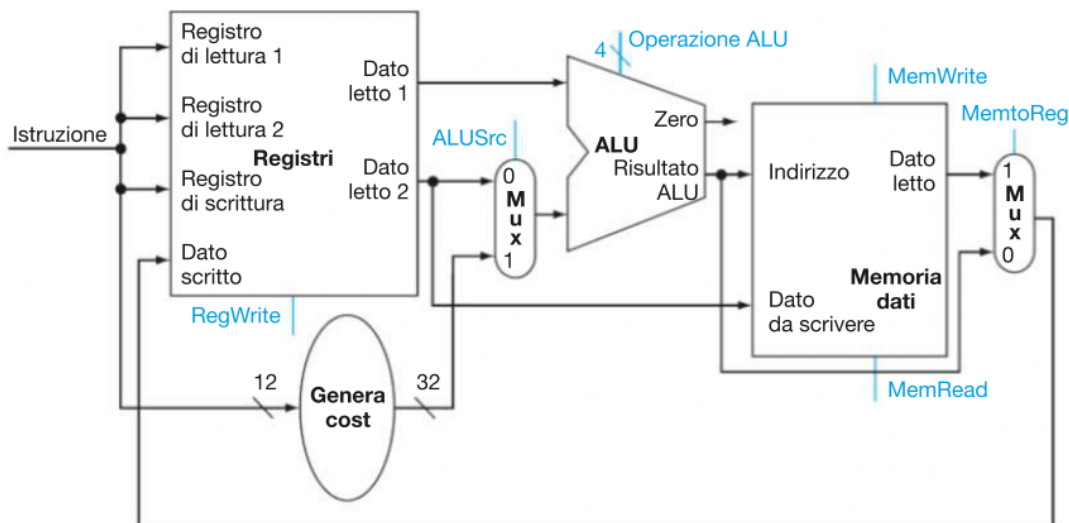


Figura 4.10 Unità di elaborazione per le istruzioni di accesso alla memoria e per le istruzioni di tipo R. Questo esempio mostra come si possa ottenere un'unica unità di elaborazione utilizzando i componenti delle Figure 4.7 e 4.8 con l'aggiunta di multiplexer. In particolare, sono necessari due multiplexer.

A questo punto, possiamo combinare tra loro tutti i pezzi introdotti finora per costruire una semplice unità di elaborazione per il nucleo dell'architettura RISC-V: possiamo aggiungere la parte relativa al prelevamento delle istruzioni (fetch, Figura 4.6), la parte di esecuzione delle istruzioni di tipo R e delle istruzioni di accesso alla memoria (Figura 4.10) e, infine, la parte dell'unità di elaborazione per i salti condizionati (Figura 4.9). La Figura 4.11 mostra l'unità di elaborazione che otteniamo dalla composizione delle diverse parti. L'istruzione di salto condizionato utilizza la ALU principale per il confronto degli operandi, contenuti in due registri; occorre perciò continuare a utilizzare il sommatore della Figura 4.9 per calcolare l'indirizzo di destinazione del salto. Un altro multiplexer consente di selezionare quale tra l'indirizzo dell'istruzione successiva ($PC + 4$) e l'indirizzo di destinazione del salto debba essere scritto nel PC.

Autovalutazione

- I. Quale delle seguenti affermazioni è corretta per un'istruzione di load (Figura 4.10)?
 - a. Il segnale MemtoReg deve essere asserito per fare in modo che il dato proveniente dalla memoria sia inviato al register file.
 - b. Il segnale MemtoReg deve essere asserito per fare in modo che il registro di destinazione corretto sia inviato al register file.
 - c. Non è importante il valore a cui viene impostato il segnale MemtoReg per le istruzioni di load.
- II. L'unità di elaborazione a singolo ciclo, descritta a grandi linee in questo paragrafo, *deve* avere due memorie separate per le istruzioni e i dati, perché:
 - a. il formato dei dati e delle istruzioni nel RISC-V è diverso e quindi occorrono due memorie distinte;
 - b. avere due memorie separate risulta meno costoso;
 - c. il processore opera in un singolo ciclo di clock e quindi non può utilizzare una memoria con una sola porta di uscita per due accessi diversi all'interno dello stesso ciclo.

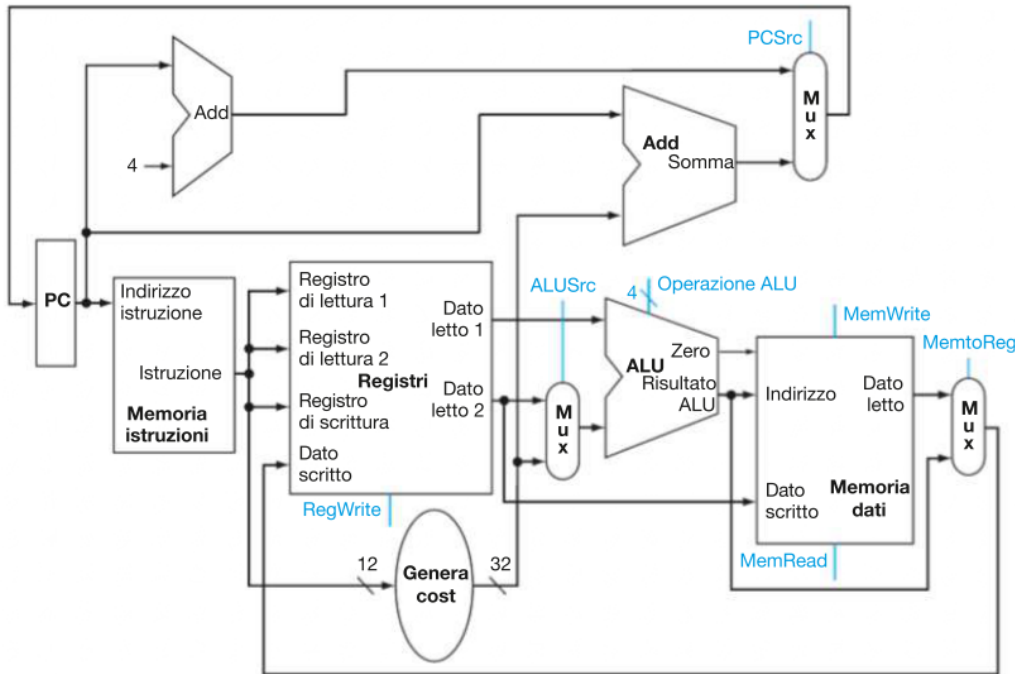


Figura 4.11 Una semplice unità di elaborazione per il nucleo dell'architettura RISC-V che combina tra loro gli elementi richiesti dalle istruzioni appartenenti ai diversi tipi. I componenti sono mostrati nelle Figure 4.6, 4.9 e 4.10. Questa unità di elaborazione può eseguire le istruzioni di base (load/store di un registro, operazioni con la ALU e salti condizionati) in un singolo ciclo di clock. Per integrare i salti condizionati si è resa necessaria solamente l'aggiunta di un altro multiplexer.

Ora che abbiamo completato quest'unità di elaborazione dati elementare, possiamo aggiungere l'unità di controllo: essa dovrà poter accettare dei valori di input e generare un segnale di scrittura per ciascun elemento di stato, un segnale di selezione per ciascun multiplexer e i segnali di controllo per la ALU. Il controllo della ALU è, per molti versi, particolare e conviene perciò progettare prima delle altre parti dell'unità di controllo.

Approfondimento. L'unità logica che genera il campo immediato deve scegliere se estendere il segno del campo immediato su 12 bit come segue. Si prendono i bit 31:20 per le istruzioni load, bit 31:25 e 11:7 per le istruzioni store o bit 31, 7, 30:25 e 11:8 per le istruzioni di salto condizionato. Dato che l'input è sempre su 32 bit per tutte le istruzioni, l'unità logica può utilizzare i bit del codice operativo dell'istruzione per selezionare il campo appropriato. Nei RISC-V il bit 6 del codice operativo è pari a 0 per le istruzioni di trasferimento dati e pari a 1 per le istruzioni di salto condizionato, il bit 5 del codice operativo è pari a 0 per le istruzioni di load e pari a 1 per le istruzioni di store. Quindi i bit 5 e 6 controllano un multiplexer 3:1, contenuto all'interno dell'unità logica di generazione del campo immediato che seleziona il gruppo di 12 bit appropriato alle istruzioni di load, store e salto condizionato.

4.4 Uno schema semplice di implementazione

In questo paragrafo esamineremo quella che può essere considerata una semplice e possibile implementazione hardware del sottoinsieme RISC-V che abbiamo scelto; utilizzeremo l'unità di elaborazione dati costruita nel paragrafo precedente, a cui aggiungeremo la relativa unità di controllo. Questa semplice implementazione consente di eseguire le istruzioni *load word* (lw), *store word* (sw), *branch if equal* (beq) e le istruzioni aritmetico-logiche add, sub, and e or.

Unità di controllo della ALU

La ALU del RISC-V descritta nell'Appendice A  prevede le 4 seguenti combinazioni dei suoi quattro input di controllo:

Linee di controllo della ALU	Operazione
0000	AND
0001	OR
0010	somma
0110	sottrazione

A seconda del tipo di istruzione, la ALU eseguirà una di queste quattro operazioni. Per le istruzioni load e store la ALU deve eseguire una somma per calcolare l'indirizzo di memoria; per le istruzioni di tipo R deve invece eseguire una delle quattro operazioni (AND, OR, somma o sottrazione) in funzione del valore dei 7 bit del campo funz7 (bit 31:25) e dei 3 bit del campo funz3 (bit 14:12) dell'istruzione (Capitolo 2). Per l'istruzione di salto condizionato, la ALU deve eseguire una sottrazione tra i due operandi e controllare se il risultato è 0.

I 4 bit di controllo della ALU possono essere generati utilizzando una piccola unità di controllo che riceve in ingresso i campi funz7 e funz3 dell'istruzione e un campo di controllo su 2 bit, chiamato ALUOp; ALUOp indica se l'operazione da eseguire è una somma (ALUOp = 00) per le istruzioni di load e store, una sottrazione (01) per le beq, oppure indica se l'operazione viene determinata dal contenuto dei campi funz7 e funz3 (10). L'uscita dell'unità di controllo della ALU è un segnale di 4 bit che controlla direttamente la ALU, generando una delle combinazioni di 4 bit mostrate nella tabella precedente.

Nella **Figura 4.12** sono riportati i valori da assegnare agli ingressi di controllo della ALU in funzione del segnale di controllo ALUOp di 2 bit e del contenuto dei campi funct7 e funct3; più avanti vedremo come l'unità di controllo principale imposti i bit di ALUOp.

L'utilizzo di livelli multipli di decodifica (in questo caso l'unità di controllo principale imposta i bit ALUOp, poi utilizzati come ingressi dell'unità di controllo della ALU che, a sua volta, genera i segnali effettivi di controllo della ALU) è una tecnica di progettazione utilizzata di frequente. Più livelli di controllo possono ridurre le dimensioni dell'unità di controllo principale; inoltre, l'uso di più unità di controllo di dimensioni ridotte può ridurre la latenza

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
lw	00	load di 1 parola	XXXXXXX	XXX	somma	0010
sw	00	store di 1 parola	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001

Figura 4.12 Il valore dei bit di controllo della ALU dipende, per le istruzioni di tipo R, dai bit di ALUOp e dai diversi codici funzione. Il codice operativo, riportato nella prima colonna, determina il valore dei bit di ALUOp. Si noti che quando il codice ALUOp vale 00 o 01, l'operazione che la ALU deve eseguire non dipende dal contenuto dei campi funz7 e funz3; in questo caso si dice che il contenuto del campo funzione è *indifferente* e viene indicato con delle X. Invece, quando ALUOp vale 10 vengono utilizzati i campi funz7 e funz3 per determinare il valore degli ingressi di controllo alla ALU. Vedi l'Appendice A  per maggiori dettagli.

dell'unità di controllo. Queste ottimizzazioni sono importanti, poiché spesso l'unità di controllo è l'elemento critico per la definizione della durata del ciclo di clock.

Ci sono diversi modi per implementare la corrispondenza tra i 2 bit del campo ALUOp e i bit dei campi funz con i 4 bit di controllo della ALU che selezionano l'operazione. Dato che interessa solamente un piccolo sottoinsieme dei possibili valori che possono essere assunti dai campi funz, e che questi ultimi vengono utilizzati solamente quando i bit di ALUOp sono pari a 10, possiamo ricorrere a un piccolo circuito logico che riconosca il sottoinsieme dei valori possibili e imposti di conseguenza i bit di controllo della ALU in modo corretto.

Come primo passo della progettazione di questo circuito, è utile costruire una *tabella della verità* che contenga le combinazioni di interesse dei campi funz e del segnale di ALUOp, come mostrato nella **Figura 4.13**. Tale **tabella della verità** mostra i 4 bit di controllo della ALU in funzione di questi tre campi in ingresso. Dato che la tabella della verità completa è molto lunga e che non siamo interessati al valore che assumono i segnali di controllo della ALU per le combinazioni che non servono (cioè, sono indifferenti), vengono riportate solamente le righe in corrispondenza delle quali deve essere specificato un preciso valore dei segnali di controllo. Nel resto di questo capitolo adotteremo la convenzione di riportare nelle tabelle della verità solamente gli elementi che devono essere asseriti, tralasciando quelli che sono non asseriti o che sono indifferenti. Questa scelta ha uno svantaggio, che sarà discusso nel Paragrafo B.2 dell'Appendice B .

Tabella della verità: utilizzata nella logica booleana, è la rappresentazione di una funzione logica. In una tabella della verità vengono elencate tutte le possibili combinazioni dei valori che possono assumere le variabili in ingresso e, per ogni combinazione, viene associata l'uscita relativa.

Termine indifferente: una variabile di ingresso di una funzione logica che non ha effetto sull'uscita della funzione. I termini indifferenti possono assumere un valore arbitrario.

Poiché in molti casi non serve conoscere i valori assunti da alcuni input, essi sono considerati **termini indifferenti**, il che consente di compattare le tabelle. Un termine indifferente di una tabella della verità, rappresentato da una X in una delle colonne associate alle variabili in ingresso, indica che l'uscita non dipende dal valore dell'ingresso associato a quella colonna. Per esempio, quando i bit di ALUOp valgono 00, come nella prima riga della tabella della Figura 4.13, i segnali di controllo della ALU vengono sempre impostati a 0010, indipendentemente dal contenuto dei campi funz; in tal caso gli ingressi dei campi funzione sono indifferenti. Incontreremo in seguito altri tipi di termini indifferenti; il lettore che non abbia familiarità con il concetto di termine indifferente è invitato a consultare l'Appendice A  per maggiori dettagli.

Una volta costruita la tabella della verità, questa può essere ottimizzata e convertita in un circuito costituito da porte logiche connesse tra loro: tale processo è di natura completamente meccanica. Perciò, invece di mostrare qui

ALUOp		Campo funz7							Campo funz3			Operazione
ALUOp1	ALUOp2	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Figura 4.13 Tabella della verità dei 4 bit di controllo della ALU (detti "operazione"). Gli ingressi della tabella della verità sono i campi ALUOp e i campi funz. Vengono riportate solamente le righe della tabella per cui i segnali di controllo della ALU devono essere asseriti. Sono mostrate anche alcune righe associate a valori indifferenti: per esempio, ALUOp non utilizza il codice 11, quindi la tabella della verità conterrà le combinazioni 1X e X1 di ALUOp anziché 10 e 01. Anche se mostriamo tutti e dieci i bit dei campi funz, notate che gli unici bit che assumono valori diversi per le quattro istruzioni di formato R sono i bit 30, 14, 13 e 12. Quindi, occorrono solamente questi quattro bit dei campi funzione come input all'unità di controllo della ALU.

questi passi, abbiamo preferito inserire la descrizione del processo di ottimizzazione e il risultato finale nel Paragrafo B.2 dell'Appendice B.

Progettazione dell'unità di controllo principale

Ora che abbiamo terminato di descrivere la progettazione di una ALU che utilizza come ingressi di controllo il campo funzione e un segnale su 2 bit, possiamo tornare ad analizzare il resto dell'unità di controllo. Iniziamo con l'identificare i diversi campi delle istruzioni e i segnali di controllo richiesti dall'unità di elaborazione mostrata nella Figura 4.11. Per capire come connettere i campi di un'istruzione all'unità di elaborazione, conviene rivedere prima il formato delle tre classi di istruzioni: tipo R, salto condizionato e load/store. Questi formati sono mostrati nella Figura 4.14.

Ci sono alcune considerazioni importanti da fare in merito a questo formato delle istruzioni, su cui ci baseremo in fase di progettazione.

- Il campo **codice operativo** o **codop** (Capitolo 2) è sempre contenuto nei bit 6:0. A seconda del codice operativo, il campo funz3 (bit 14:12) e funz7 (31:25) servono come campi di estensione del codice operativo.
- Il primo registro operando si trova sempre nei bit in posizione 19:15 (rs1) per le istruzioni di tipo R e le istruzioni di salto condizionato. Questo campo specifica anche il registro base per le istruzioni di load e di store.
- Il secondo registro operando si trova sempre nei bit in posizione 24:20 (rs2) per le istruzioni di tipo R e le istruzioni di salto condizionato. Questo campo specifica anche il registro il cui contenuto viene copiato in memoria nelle istruzioni di store.
- Il secondo operando può anche essere costituito dai 12 bit di offset delle istruzioni di branch o di load/store.
- Il registro destinazione si trova sempre nei bit in posizione 11:7 (rd) per le istruzioni di tipo R e di load.

Codice operativo (codop): il campo che denota il tipo di operazione e il formato di un'istruzione.

Il primo principio di progettazione introdotto nel Capitolo 2 – *La semplicità favorisce la regolarità* – risulta particolarmente utile nella progettazione dell'unità di controllo dell'unità di elaborazione.

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediato[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop

Figura 4.14 Le quattro classi di istruzioni (tipo R, load, store e salto condizionato) utilizzano quattro diversi formati di istruzione. (a) Formato delle istruzioni di tipo R (codice operativo = 51_{dec}). Queste hanno tre operandi contenuti nei registri: rs1, rs2 e rd; i campi rs1 e rs2 contengono il numero dei due registri sorgenti e rd contiene il numero del registro destinazione. L'operazione che la ALU deve eseguire è codificata nei campi funz3 e funz7 e verrà decodificata dall'unità di controllo della ALU, progettata nel paragrafo precedente. Le istruzioni di tipo R implementate sono add, sub, and e or. (b) Formato di tipo I delle istruzioni di load (codice operativo = 3_{dec}) di tipo I. Il registro rs1 è il registro base il cui contenuto viene sommato al campo immediato di 12 bit per ottenere l'indirizzo del dato in memoria. Il campo rd è il registro destinazione per il valore letto dalla memoria. (c) Formato di tipo S delle istruzioni di store (codice operativo = 35_{dec}) di tipo S. Il registro rs1 è il registro base il cui contenuto viene sommato al campo immediato di 12 bit per ottenere l'indirizzo del dato in memoria (il campo immediato è suddiviso in un gruppo di 7 e in un gruppo di 5 bit). Il campo rs2 è il registro sorgente il cui valore viene copiato nella memoria. (d) Formato di tipo SB delle istruzioni branch if equal (codice operativo = 99_{dec}). I registri rs1 e rs2 vengono confrontati. Il campo indirizzo immediato di 12 bit viene preso, il suo bit di segno esteso, fatto scorrere a sinistra di una posizione e sommato al PC per calcolare l'indirizzo di destinazione del salto. Le Figure 4.17 e 4.18 forniscono il razionale di questo insolito posizionamento dei bit nel formato SB.

Interfaccia hardware/software

Rispetto al MIPS, i formati delle istruzioni del RISC-V sembrano più complicati ma in realtà semplificano l'hardware e possono perfino diminuire la durata del periodo di clock in alcune implementazioni RISC-V, specialmente quelle in pipeline che vedremo nel Paragrafo 4.6. Dato che i compilatori, gli assembleri e i debugger nascondono ai programmatori i dettagli del formato delle istruzioni, perché non aiutare l'hardware? Il primo esempio è il formato dell'istruzione di store. La Figura 4.15 mostra i formati delle istruzioni MIPS per il trasferimento di dati con la memoria e di quelle aritmetiche e il loro impatto sul cammino di esecuzione. Il MIPS richiede un multiplexer 2:1 per specificare su quale registro scrivere il dato letto dalla memoria, che in questo caso non serve. Questo multiplexer può trovarsi facilmente sul cammino critico e quindi aumenterebbe il periodo di clock. Per mantenere sempre il registro destinazione nei bit da 7 a 11 per tutte le istruzioni, il formato S del RISC-V ha dovuto suddividere il campo immediato in due gruppi di bit: i bit da 25 a 31 contengono il campo immediato[11:5] e i bit da 7 a 11 il campo immediato[4:0]. Sembra strano, confrontando la gestione del campo immediato con quella del MIPS nel quale i bit del campo immediato sono tutti contigui, ma l'assemblatore del RISC-V nasconde questa complessità e l'hardware ne beneficia.

Il secondo esempio sembra ancora più strano. La Figura 4.16 mostra che il RISC-V ha due formati, SB e UJ, i quali hanno gli stessi campi della stessa dimensione dei formati S e U e contengono anch'essi una costante, con la differenza che i bit sono scambiati. Anche i formati SB e UJ semplificano l'hardware procurando un lavoro maggiore all'assemblatore. Le figure successive mostrano come l'hardware genera il campo immediato nel RISC-V. La Figura 4.17 mostra quali bit dell'istruzione corrisponderebbero ai bit del campo immediato di questi tipi di istruzioni, se le istruzioni di salto condizionato utilizzassero il formato S, invece del formato SB, e se le istruzioni di salto incondizionato utilizzassero il formato U, invece del formato UJ. L'ultima riga mostra il numero di input per ogni bit di output, che determina il numero di porte del multiplexer utilizzato per generare il campo immediato.

La Figura 4.18 mostra invece il formato vero delle istruzioni di salto condizionato e incondizionato e il numero ridotto di bit specifici. I formati SB e UJ riducono la dimensione del multiplexer per i bit da 12 a 19 del campo immediato da 3:1 a 2:1 e per i bit da 10 a 1 del campo immediato da 4:1 a 2:1. Anche qui, i progettisti del RISC-V hanno definito dei formati che appaiono strani ma che sono efficienti, semplificando i multiplexer 18:1. Il risparmio non è significativo per i processori di fascia alta, ma molto prezioso per i processori di fascia bassa e tutto il costo è supportato dagli assembleri (e dagli autori).

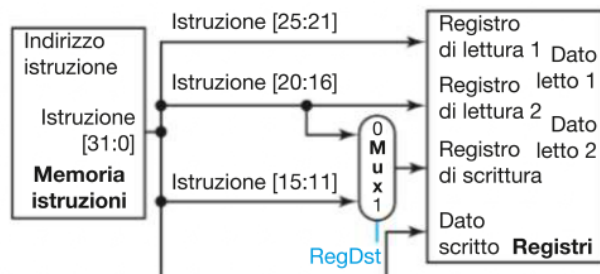
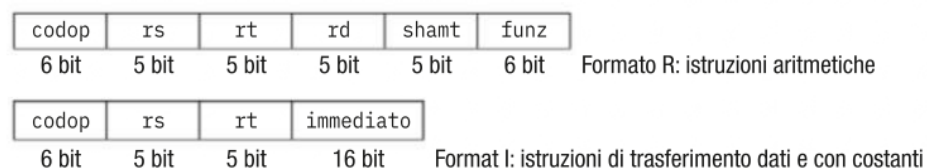


Figura 4.15 Il MIPS, il formato delle istruzioni aritmetiche e di quelle di trasferimento dei dati, e il loro impatto sul cammino di esecuzione del MIPS. Nelle istruzioni aritmetiche MIPS che utilizzano il formato R, rd è il registro destinazione, rs il registro del primo operando e rt quello del secondo operando. Nelle istruzioni di load e che utilizzano una costante, rs contiene sempre il primo operando, ma rt diventa il registro di scrittura. Da qui nasce l'esigenza di un multiplexer 2:1 per scegliere tra rt e rd il numero del registro corretto su cui scrivere il risultato.

Nome (Dimensione campo)	Campo						Commenti
	7 bit	5 bit	5 bit	3 bit	5 bit	7 bit	
Tipo R	funz7	rs2	rs1	funz3	rd	codop	Formato istruzioni aritmetiche
Tipo I	imm[11:0]		rs1	funz3	rd	codop	Load e aritmetiche con costante
Tipo S	imm[11:5]	rs2	rs1	funz3	imm[4:0]	codop	Store
Tipo SB	imm[12,10:5]	rs2	rs1	funz3	imm[4:1,11]	codop	Salti cond
Tipo UJ	imm[20, 10:1, 11, 19:12]				rd	codop	Formato salti incond
Tipo U	imm[31:12]				rd	codop	Formato upper immed

Figura 4.16 I formati RISC-V. La Figura 4.16 introduce i formati R, I, S e U, che sono intuitivi.

		Campo immediato generato bit per bit																																			
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Istruzione	Formato	Input del campo immediato bit per bit																																			
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20				
Store	S	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	i11	i10	i9	i8	i7			
Salto cond	S	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	i30	i29	i28	i27	i26	i25	i24	"	"	"	"	0			
Salto incond	U	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	"	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	i19	i18	i17	i16	i15	i14	i13	i12	"
Load Upper imm	U	"	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	i19	i18	i17	i16	i15	i14	i13	i12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	"	
Num input		1	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3	3	3	3	3	4	4	4	4	4	4	4	4	4	4	4	4	4	3		

Figura 4.17 Gli input del campo immediato (costante) quando i salti condizionati utilizzano il formato S e i salti incondizionati il formato U.

		Campo immediato generato bit per bit																																
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Istruzione	Formato	Input del campo immediato bit per bit																																
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	
Store	S	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	i11	i10	i9	i8	i7
Salto cond	SB	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	0
Salto incond	UJ	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	i19	i18	i17	i16	i15	i14	i13	i12	i20	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc	cc
Load Upper imm	U	cc	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	cc	cc	cc	cc	cc	cc	cc	cc	0	0	0	0	0	0	0	0	0	0	0	0	cc
Num input		1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	2	2	2	2	2	2	3	3	3	3	3	

Figura 4.18 Gli input del campo immediato (costante) quando i salti condizionati utilizzano il formato SB e i salti incondizionati il formato UJ, che sono quelli che utilizza il RISC-V.

Utilizzando le osservazioni riportate sopra, possiamo ora aggiungere all'unità di elaborazione elementare che abbiamo costruito le etichette sui campi dell'istruzione. La **Figura 4.19** mostra queste aggiunte, un blocco che rappresenta l'unità di controllo della ALU, i segnali di scrittura degli elementi di stato, il segnale di lettura della memoria dati e i segnali di controllo dei multiplexer. Dato che tutti i multiplexer hanno due ingressi, ciascuno di essi richiede un solo segnale di controllo.

La **Figura 4.19** mostra i sei segnali di controllo di 1 bit ciascuno e il segnale di controllo ALUOp di 2 bit. Abbiamo già spiegato come funziona questo segnale, ed è utile definire informalmente come debbano operare gli altri segnali di controllo prima di determinare il modo in cui devono essere impostati durante l'esecuzione delle istruzioni. La **Figura 4.20** descrive il funzionamento di questi sei segnali di controllo.

Ora che abbiamo visto il funzionamento di ciascuno dei segnali di controllo, possiamo passare a esaminare come debbano essere impostati. L'unità di controllo può impostare tutti i segnali tranne uno, basandosi esclusivamente sul codice operativo dell'istruzione stessa. L'eccezione è costituita dal segnale di controllo PCSrc: esso deve essere asserito se l'istruzione è una branch

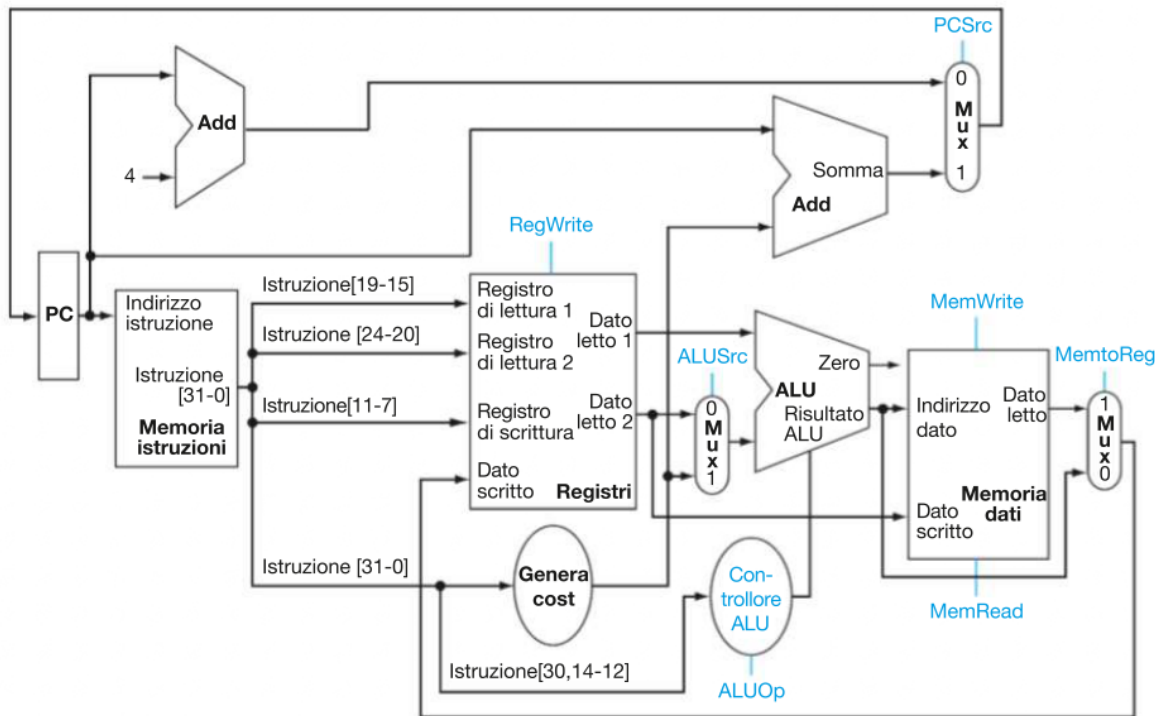


Figura 4.19 L'unità di elaborazione della Figura 4.11 con tutti i multiplexer e i segnali di controllo necessari. I segnali di controllo sono mostrati in blu. È stato introdotto anche il blocco che rappresenta il controllore della ALU. Il PC non richiede un segnale di controllo della scrittura esplicito, dal momento che viene scritto al termine di ogni ciclo di clock; la logica di controllo dei salti determina se nel PC debba essere scritto il valore precedente del PC incrementato di 4 oppure l'indirizzo di destinazione del salto.

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nulla	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di PC + 4	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nulla	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nulla	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

Figura 4.20 Effetto di ciascuno dei sette segnali di controllo. Quando il segnale di controllo a 1 bit di un multiplexer a due vie è asserito, il multiplexer seleziona l'ingresso etichettato con 1; in caso contrario, cioè se il segnale di controllo non è asserito, il multiplexer seleziona l'ingresso etichettato con 0. Si ricordi che tutti gli elementi di stato ricevono il clock come ingresso implicito e che il clock controlla le operazioni di scrittura. Far commutare il clock mediante un segnale esterno a un elemento di stato potrebbe causare problemi di temporizzazione (vedi Appendice A per la discussione di questo problema).

if equal, decisione che l'unità di controllo è in grado di prendere, *ma anche* se l'uscita Zero della ALU, utilizzata per il confronto di uguaglianza, è vera. Per generare il segnale PCSrc occorre dunque collegare in AND un segnale proveniente dall'unità di controllo, che verrà chiamato *Branch*, con l'uscita Zero della ALU.

Questi otto segnali di controllo, i sei della Figura 4.20 e i due di ALUOp, possono ora essere impostati sulla base dei segnali di ingresso all'unità di controllo, che sono i sette bit del codice operativo (bit 6:0). La **Figura 4.21** mostra l'unità di elaborazione completa, con l'unità di controllo e i relativi segnali.

Prima di iniziare a scrivere un insieme di equazioni o una tabella della verità per specificare il modo di operare dell'unità di controllo, è utile cercare di definire informalmente il suo funzionamento. Dato che il valore di ciascuna linea di controllo dipende solamente dal codice operativo, possiamo dire se ciascun segnale debba valere 0, 1 o sia indifferente (X) per i diversi valori che il codice operativo può assumere. La **Figura 4.22** riporta il valore di ciascun segnale di controllo in funzione del valore del codice operativo, ed è ottenuta direttamente dalle informazioni contenute nelle Figure 4.12, 4.20 e 4.21.

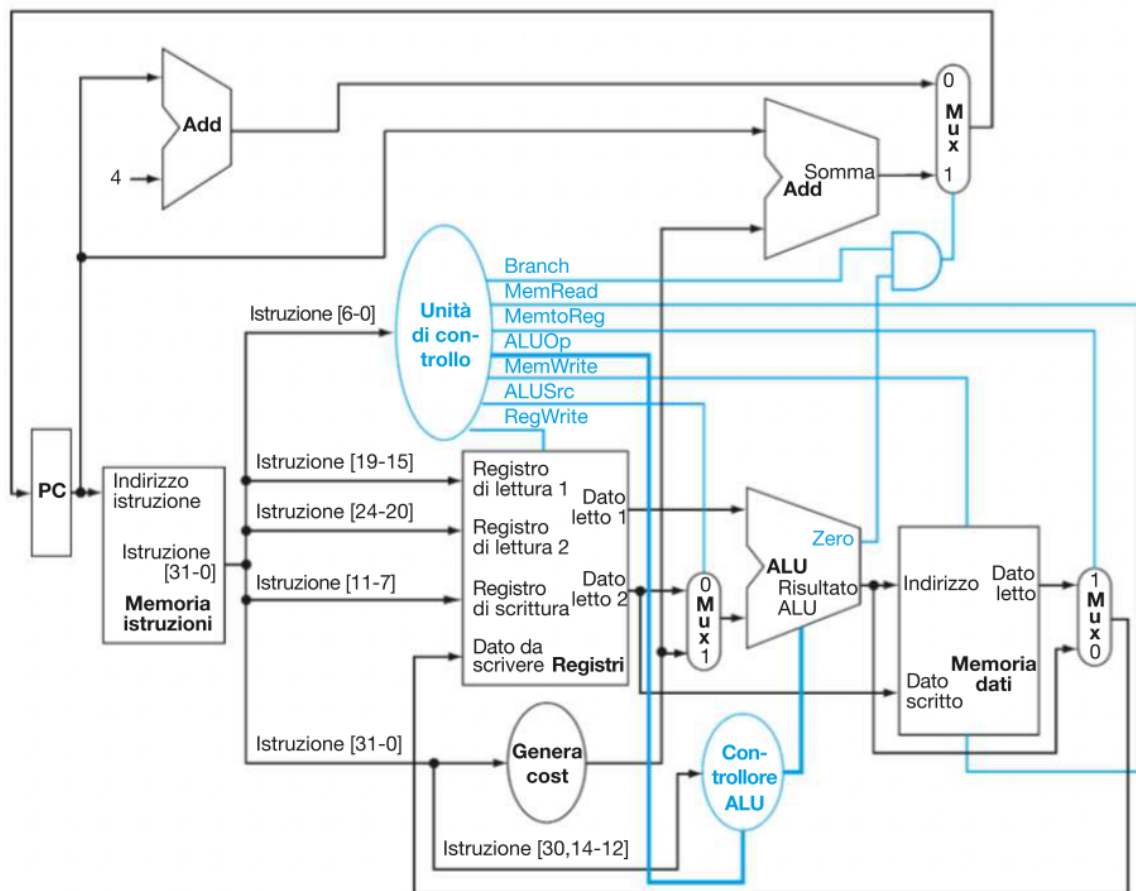


Figura 4.21 L'unità di elaborazione costruita finora, con la sua unità di controllo. L'input dell'unità di controllo è il campo a 7 bit dell'istruzione che costituisce il codice operativo. L'uscita dell'unità di controllo è formata da due segnali a 1 bit utilizzati per controllare i multiplexer (ALUSrc e MemtoReg), tre segnali a 1 bit per controllare lettura e scrittura dei register file e della memoria dati (RegWrite, MemRead e MemWrite), un segnale a 1 bit utilizzato come segnale di controllo per i salti condizionati (Branch) e un segnale di controllo a 2 bit per la ALU (ALUOp). Una porta AND viene utilizzata per combinare il segnale di Branch e l'uscita Zero della ALU; l'uscita di questa porta determina da dove prendere il valore successivo del PC e genera il segnale di controllo PCSrc per il multiplexer posizionato in alto (Figura 4.15). Si noti che PCSrc viene derivato e non proviene direttamente dall'unità di controllo; per questo motivo, ometteremo di indicarlo nelle figure successive.

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Figura 4.22 Il valore assunto dalle linee di controllo è completamente determinato dal campo del codice operativo dell'istruzione. La prima riga della tabella corrisponde alle istruzioni in formato R (add, sub, and e or), per le quali i campi registro sorgente sono rs1 e rs2 e il campo registro destinazione è rd; questo stabilisce come impostare il segnale ALUSrc. Inoltre, le istruzioni di tipo R scrivono un registro (RegWrite = 1), ma non leggono né scrivono nella memoria dati. Quando il segnale di controllo Branch è impostato a 0, il PC viene aggiornato incondizionatamente a PC + 4; altrimenti, il contenuto del PC viene sostituito con l'indirizzo della destinazione del salto, ma solo quando anche l'uscita Zero della ALU è alta. Il campo ALUOp per le istruzioni di tipo R è impostato a 10 per indicare che i segnali di controllo della ALU dovranno essere generati a partire dai campi funz. La seconda e la terza riga di questa tabella riportano l'impostazione dei segnali di controllo per lw e sw, per le quali ALUSrc e ALUOp assumono il valore richiesto perché venga calcolato l'indirizzo in memoria, mentre MemRead e MemWrite determinano il tipo di accesso alla memoria. Infine, in una load, RegWrite viene impostato in modo tale che il dato letto dalla memoria venga scritto nel registro rd. Nell'istruzione di salto condizionato il campo ALUOp indica la sottrazione (01), utilizzata per controllare l'uguaglianza degli operandi. Si noti che il segnale MemtoReg risulta irrilevante quando il segnale RegWrite è uguale a 0: dal momento che l'istruzione non scrive nel register file, il valore del dato presente sulla porta di scrittura non viene utilizzato; quindi il valore di MemtoReg nelle ultime due righe viene sostituito con una X per indicare che è indifferente. La scelta di impostare questi segnali di controllo con un valore indifferente deve essere fatta dal progettista, poiché dipende dalla conoscenza del funzionamento dell'unità di elaborazione.

Funzionamento dell'unità di elaborazione

Con le informazioni contenute nelle Figure 4.20 e 4.22 potremmo progettare la logica dell'unità di controllo; tuttavia, prima di procedere esaminiamo l'utilizzo che ciascuna istruzione fa dell'unità di elaborazione. Nelle prossime figure sarà riportato il flusso di elaborazione attraverso l'unità di elaborazione di tre istruzioni appartenenti alle tre diverse classi. I segnali di controllo asseriti e gli elementi attivi dell'unità di elaborazione saranno evidenziati. Si noti che un multiplexer il cui segnale di controllo valga 0 compie un'azione ben definita, anche se il suo segnale di controllo non viene evidenziato. I segnali a più bit saranno evidenziati se risulta asserito almeno uno dei segnali associati ai singoli bit.

La Figura 4.23 mostra il modo in cui l'unità di elaborazione esegue un'istruzione di tipo R, per esempio add x1, x2, x3. Sebbene l'esecuzione avvenga in un singolo ciclo di clock, possiamo pensare che l'esecuzione proceda attraverso una sequenza di quattro passi, che possiamo identificare analizzando il flusso delle informazioni:

1. L'istruzione viene prelevata dalla memoria e si incrementa il PC;
2. I due registri x2 e x3 vengono letti dal register file, mentre l'unità di controllo principale calcola il valore da attribuire alle linee di controllo;
3. La ALU elabora i dati letti dal register file, utilizzando alcuni bit del codice operativo per selezionare l'operazione della ALU;
4. Il risultato calcolato dalla ALU viene scritto nel registro destinazione (x1) del register file.

In modo analogo, si può illustrare l'esecuzione di un'istruzione di caricamento di un registro, per esempio

```
lw x1, offset(x2)
```

utilizzando lo stesso approccio della Figura 4.23. Le unità funzionali attive e i segnali di controllo asseriti per l'elaborazione di un'istruzione di load sono mostrati nella Figura 4.24. Così come avevamo scomposto l'istruzione di tipo R in quattro passi sequenziali, possiamo scomporre l'istruzione di load in cinque passi sequenziali:

1. L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC;
2. Viene letto il contenuto di un registro (x2) dal register file;

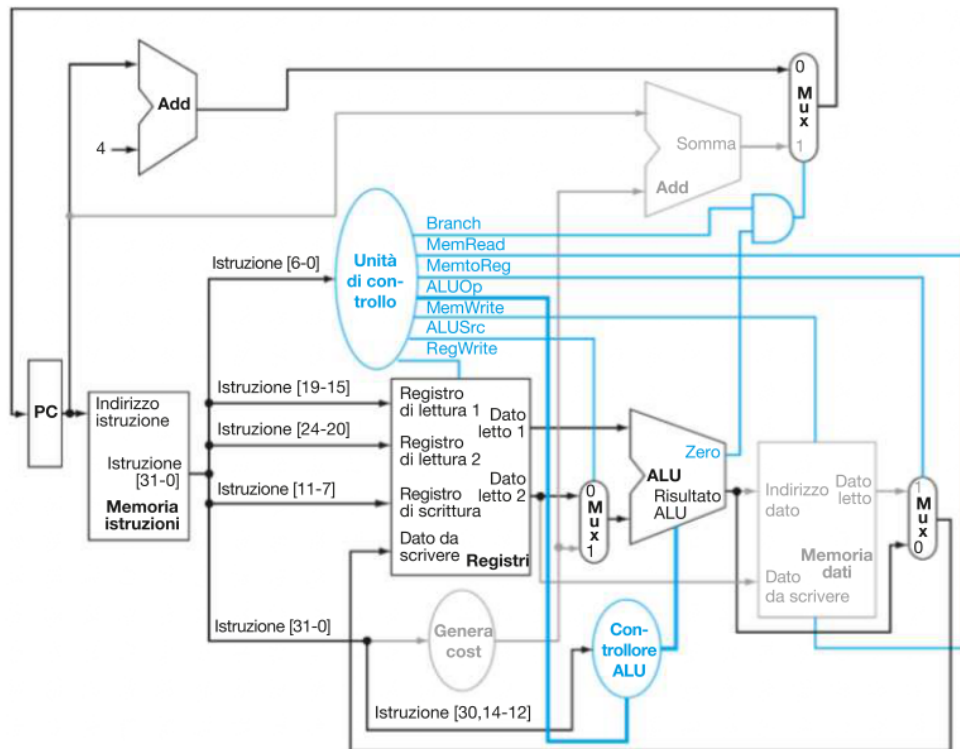


Figura 4.23 L'unità di elaborazione durante l'esecuzione di un'istruzione di tipo R, come `add x1, x2, x3`. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu.

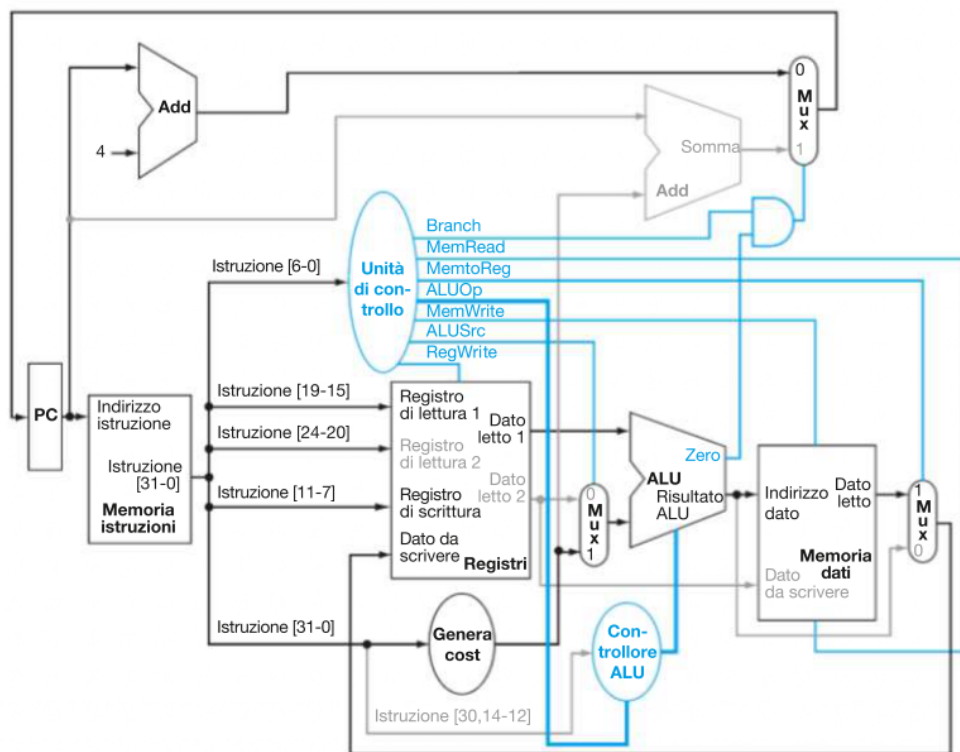


Figura 4.24 L'unità di elaborazione durante l'esecuzione di un'istruzione di tipo `load`. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu. Un'istruzione di `store` viene eseguita in modo molto simile; le differenze principali sono tre: i segnali di controllo della memoria devono specificare una scrittura (del contenuto del secondo registro letto), il secondo registro letto contiene il dato da scrivere in memoria e non è presente l'operazione di scrittura del dato nel register file.

3. La ALU somma il valore letto dal register file ai 12 bit del campo offset (campo immediato) dell'istruzione, considerati dotati di segno ed estesi a 32 bit;
4. La somma calcolata dalla ALU viene utilizzata come indirizzo per la memoria dati;
5. Il dato proveniente dall'unità di memoria dati viene scritto nel register file nel registro x1.

Infine, possiamo utilizzare la stessa tecnica per esaminare l'esecuzione di un'istruzione di salto condizionato, per esempio `beq x1, x2, offset`. Essa si comporta in modo molto simile a un'istruzione di tipo R, con la differenza che l'uscita della ALU viene utilizzata per scegliere se scrivere nel PC il valore `PC + 4` oppure l'indirizzo di destinazione del salto. I quattro passi sequenziali dell'esecuzione di una `beq` sono mostrati nella **Figura 4.25**:

1. L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC;
2. Vengono letti dal register file i due registri x1 e x2;
3. La ALU esegue la sottrazione del contenuto dei due registri letti dal register file; il valore del PC viene sommato ai 12 bit del campo offset dell'istruzione, considerati dotati di segno, estesi a 32 bit e fatti scorrere di una posizione a sinistra; il risultato costituisce l'indirizzo di destinazione del salto;
4. La linea Zero in uscita dalla ALU viene utilizzata per determinare da quale sommatore prendere l'indirizzo successivo da scrivere nel PC.

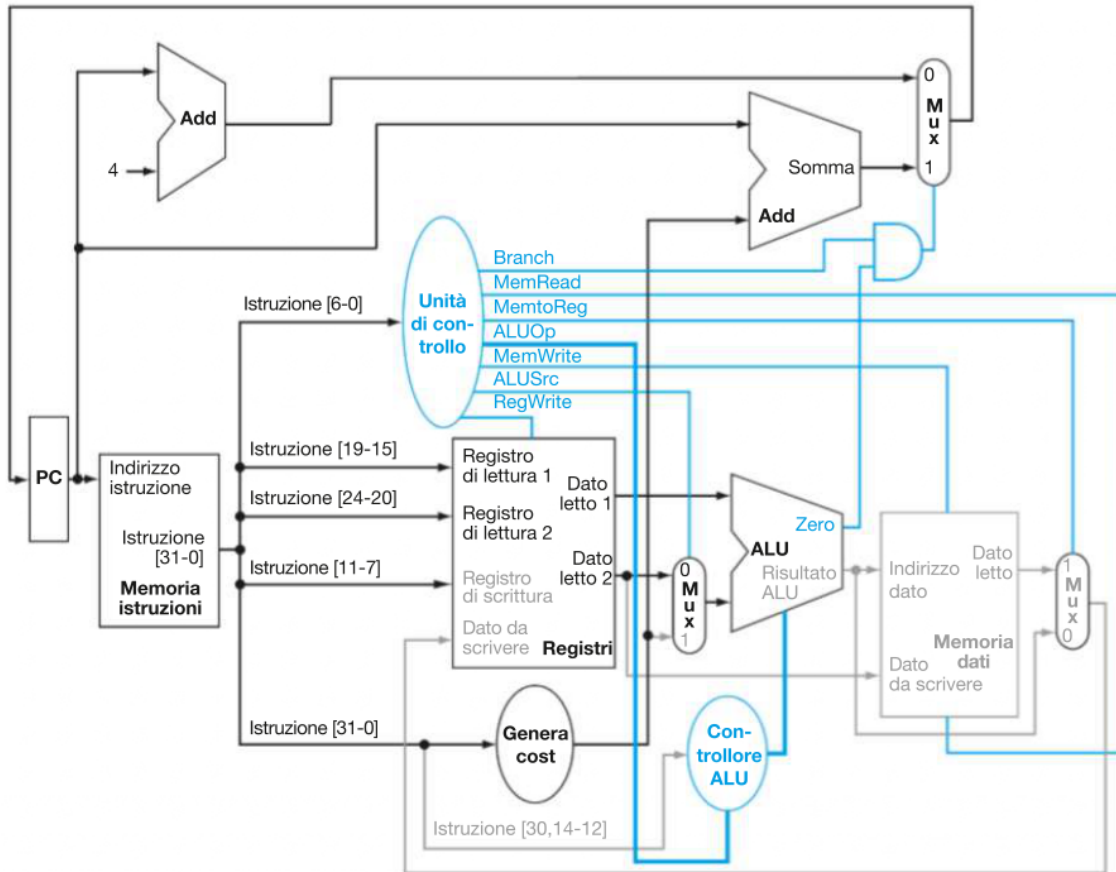


Figura 4.25 L'unità di elaborazione durante l'esecuzione di un'istruzione **branch equal**. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu. Dopo aver utilizzato il register file e la ALU per eseguire il confronto, l'uscita Zero della ALU viene impiegata per scegliere il valore successivo del program counter tra i due possibili candidati.

Input o output	Nome del segnale	Formato R	ld	sd	beq
Input	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Output	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp01	1	0	0	0
	ALUOp02	0	0	0	1

Figura 4.26 La funzione logica dell'unità di controllo per l'implementazione a singolo ciclo di clock è completamente specificata da questa tabella della verità. Le prime sette righe della tabella riporta le combinazioni del codice operativo associate alle quattro classi di istruzioni esaminate, una per ogni colonna; i segnali di controllo vengono impostati in funzione di questi ingressi. La parte inferiore della tabella fornisce invece le uscite in funzione del codice operativo. Quindi l'uscita RegWrite viene asserita in corrispondenza di due diverse combinazioni degli input. Se si considerano solamente i quattro codici operativi riportati, la tabella della verità si può semplificare utilizzando la condizione di indifferenza per alcuni ingressi: per esempio, si può identificare un'istruzione in formato R con l'espressione $Op4 \cdot Op5$, dato che questa è sufficiente a distinguere le istruzioni in formato R dalle lw, sw e beq. Tuttavia non sfrutteremo queste semplificazioni, dato che in un'implementazione completa dell'insieme delle istruzioni anche i restanti codici operativi vengono utilizzati nell'implementazione completa del RISC-V.

Completamento dell'unità di controllo

Dopo aver esaminato i diversi passi di esecuzione delle istruzioni, possiamo procedere all'implementazione dell'unità di controllo. La funzione di controllo può essere determinata in modo esatto a partire dal contenuto della Figura 4.22. Le uscite dell'unità di controllo sono i segnali di controllo, mentre i suoi input sono costituiti dai 7 bit del codice operativo. Possiamo quindi scrivere una tabella della verità per ciascuna delle uscite, basata sulla codifica binaria dei codici operativi. La Figura 4.26 mostra la funzione logica implementata nell'unità di controllo sotto forma di una grande tabella della verità che contiene tutte le uscite e i bit del codice operativo come ingressi. Tale tabella specifica completamente la funzione di controllo e può essere implementata direttamente mediante porte logiche in modo automatico. Quest'ultimo passaggio viene descritto nel Paragrafo C.2 dell'Appendice C.

Perché oggi non si utilizzano più implementazioni a singolo ciclo?

Sebbene un processore a singolo ciclo funzioni correttamente, esso non viene più utilizzato nelle implementazioni moderne, perché troppo inefficiente. Per capirne il motivo, si noti che, in questa implementazione, il periodo di clock deve avere la stessa durata per tutte le istruzioni; quindi, il periodo di clock è determinato dal cammino di esecuzione più lungo all'interno del processore (cammino critico). Tale cammino sarà molto probabilmente associato a un'istruzione di load, dato che essa utilizza cinque unità funzionali in sequenza: la memoria istruzioni, il register file, la ALU, la memoria dati e, di nuovo, il register file. Anche se il CPI è pari a 1 (Capitolo 1), le prestazioni complessive di un'implementazione a singolo ciclo saranno probabilmente scarse, poiché la durata del periodo di clock risulterà troppo lunga.

Il prezzo da pagare per un'implementazione a singolo ciclo con una durata del clock fissa è significativo, ma potrebbe comunque essere considerato

accettabile per un insieme limitato di istruzioni, come quello considerato. I primi calcolatori, dotati di un insieme di istruzioni semplice, utilizzavano questa implementazione. Tuttavia, se si volesse implementare un'unità di elaborazione in virgola mobile o un insieme di istruzioni comprendente istruzioni più complesse, un progetto a singolo clock non funzionerebbe altrettanto bene.

Dato che dobbiamo assicurare che la durata del ciclo di clock sia pari alla durata dell'istruzione più lunga da eseguire, non si possono sfruttare quelle tecniche che consentono di ridurre il tempo di esecuzione delle istruzioni più comuni ma che non riducono la durata dell'esecuzione dell'istruzione più lenta: un'implementazione a un solo ciclo è in contrasto quindi con una delle grandi idee introdotte nel Capitolo 2, che richiede di **rendere veloci le situazioni più comuni**.



SITUAZIONI COMUNI

Autovalutazione

Nel Paragrafo 4.6 illustreremo una tecnica di implementazione diversa, chiamata pipeline. Essa utilizza un'unità di elaborazione simile a quella a singolo ciclo ma molto più efficace, perché consente un throughput sensibilmente maggiore. Le pipeline sono più efficienti grazie alla sovrapposizione temporale dell'esecuzione di diverse istruzioni. Osserviamo i segnali di controllo nella Figura 4.26. Si possono combinare fra loro? Esiste qualche segnale di controllo che può essere sostituito con l'inverso di un altro? (Suggerimento: si prendano in considerazione i segnali indifferenti.) Se sì, è possibile sostituire un segnale di controllo con un altro senza utilizzare un invertitore?

4.5 Un'implementazione multi-ciclo

Nel paragrafo precedente, abbiamo suddiviso l'esecuzione di ciascuna istruzione in una sequenza di passi che corrispondono alle unità funzionali richieste a ogni passo. Possiamo utilizzare questa suddivisione per creare un'**implementazione multi-ciclo**. In un'implementazione multi-ciclo, ciascun passo dell'esecuzione richiederà 1 ciclo di clock e sarà consentito utilizzare le unità funzionali più di una volta per ogni istruzione, a patto che vengano utilizzate in cicli di clock diversi. Questa condivisione delle unità funzionali consente di ridurre la quantità di hardware richiesto. La possibilità di eseguire le istruzioni in un numero diverso di cicli di clock e la condivisione delle unità funzionali all'interno dell'esecuzione della singola istruzione sono il maggiore vantaggio dell'implementazione multi-ciclo del MIPS. Questo paragrafo, disponibile on-line, descrive l'implementazione multi-ciclo del MIPS.

Sebbene questa implementazione possa ridurre i costi dell'hardware, quasi tutti i chip oggi implementano la pipeline, che non ha l'obiettivo di aumentare le prestazioni sulla singola istruzione rispetto a quella a singolo ciclo di clock. Per questo motivo, i lettori non interessati a questo argomento, possono saltare l'implementazione multi-ciclo e andare direttamente alla pipeline. Tuttavia, dato che alcuni docenti vedono un vantaggio didattico nello spiegare l'implementazione multi-ciclo prima della pipeline, la descrizione di questa implementazione viene offerta on-line.

4.6 Introduzione alla pipeline

La **pipeline** è una tecnica di implementazione hardware di un insieme di istruzioni che prevede la sovrapposizione temporale dell'esecuzione delle diverse istruzioni; attualmente è utilizzata quasi dalla totalità dei calcolatori.

Il tempo non va sprecato.

Proverbio americano